SHRINCS specification

Blockstream Research

April 9, 2026

Abstract

This document specifies **SHRINCS** (Shrunken SHPINCS), a post-quantum signature scheme optimized for Bitcoin and Liquid. SHRINCS combines efficient stateful hash-based signatures (unbalanced XMSS with WOTS+C) for normal operation with a stateless fallback (an SPHINCS+ variant) for scenarios where state management is corrupted or unavailable. This specification provides with two parameter sets optimized for different deployment scenarios: (1) SHRINCS-B (Bitcoin-optimized) minimizes signature size for byte-based fee systems (**324**-byte signature with 2043 hashes for verification for the first use, growing by 16 bytes and one hash per subsequent signature(except that the last two stateful signatures)); (2) SHRINCS-L (Liquid-optimized) reduces verification cost for computation-based fee systems (only **55** hash operations with 1092-byte signature for the first use). Both variants support "stateless-only after recovery" mode to preserve funds while accepting larger and more expensive from the verification perspective signatures.

SHRINCS Team and Contributors: *Mikhail Kudinov, Dmitrii Kurbatov, Oleksandr Kurbatov, Jonas Nick*

Contents

1 Motivation

Current Bitcoin signature schemes (ECDSA[Nat13] and Schnorr[WNR20] over secp256k1) are vulnerable to quantum attacks via Shor's algorithm[Sho94; Sho97]. Hash-based signature schemes offer post-quantum security relying only on the security of cryptographic hash functions, which are believed to resist quantum attacks apart from Grover's quadratic speedup (mitigated by doubling hash output lengths).

The key challenges for deploying hash-based signatures in Bitcoin are:

1. **Signature size**: NIST-standardized SLH-DSA signatures [Nat24] range from 7KB to 50KB, significantly larger than current 64-byte Schnorr signatures. This increases transaction fees and reduces the network's throughput.

2. **Stateful vs. Stateless tradeoff**: Stateful schemes (e.g., XMSS [Coo+20]) offer compact signatures but require careful state management to prevent key reuse. Additionally, Bitcoin users expect to restore wallets from static seed phrases (BIP-39[Pal+13]). Pure stateful schemes break this model since signing state cannot be recovered from a seed alone, the signer must know which one-time keys have already been used. Stateless schemes (e.g., SLH-DSA) are robust but produce large signatures.

1.1 SHRINCS

SHRINCS (Shrunken SHPINCS) is a hybrid post-quantum signature scheme that combines the efficiency of stateful signatures with the robustness of stateless ones. It provides with two parameters set depending on what matters for verifier:

- SHRINCS-B (Bitcoin-optimized): Minimizes signature size at the cost of more verification hashes.
- SHRINCS-L (Liquid-optimized): Minimizes verification cost at the cost of larger signatures.

The advantages of SHRINCS are following:

- **Efficient stateful path.** During normal operation with intact state, SHRINCS uses an Unbalanced XMSS (UXMSS) tree with WOTS+C one-time signatures. Signature sizes and verification costs are:
  - SHRINCS-B: **only** `308 + 16q` bytes and `2042 + q` hashes, which makes this setting practical from the *witness size* perspective
  - SHRINCS-L: `1076 + 16q` bytes and **only** `54 + q` hashes, which makes this setting practical from the *verification complexity* perspective

  bytes (where `q >= 1` is the signature counter).
- **Robust stateless fallback.** When the state is lost, corrupted, or exhausted, SHRINCS falls back to a SPHINCS+-based stateless signature (2568 or 4104 bytes). This is still smaller than standard SLH-DSA.
- **Static seed backup.** Users can back up their wallet using a standard seed phrase. Upon recovery, the wallet operates in stateless-only mode, preserving funds while accepting larger signatures.
- **The number of supported signatures**. The construction supports up to 141 (SHRINCS-B) and 189 (SHRINCS-L) stateful signatures and up to 2^{20} stateless signatures, which should be enough for wallets.

1.2 Limitations and Practical Considerations

Users and developers should be aware of the following limitations inherent to stateful signature schemes.

State Management Across Devices. The signing state (specifically, the counter `q` indicating which one-time keys have been already used) cannot be safely shared across multiple devices without coordination. Two devices signing with the same `q` value would reuse a one-time signature, which leads to key compromise. Some mitigation strategies can be:

- Dedicate disjoint ranges of leaf indices to different devices.
- Use a single signing device with secure state storage (other devices are stateless backups).
- Accept stateless-only operation for multi-device wallets.

State Corruption and Power Failure. If a signing operation is interrupted (e.g., due to power failure) after producing a signature but before persisting the updated state, the signer may not know whether the one-time key was used. Subsequent signing attempts risk catastrophic key reuse. Approaches to mitigate:

- Incrementing the state counter before the signature. If signing fails, a leaf is wasted but security is preserved (we utilize this approach in the specification).
- State corruption detection, to maintain checksums or redundant state copies to detect corruption. If corruption is detected, fall back to stateless mode.
- Other state and backup risks and management techniques are described in [Wig+26].

State Growth with Multiple Addresses. Each SHRINCS-based address requires its own signing state. As users generate new addresses (as recommended for privacy), they must maintain state for each address that may be used for future spending. This state must be preserved for as long as the address holds funds. Mitigation:

- Wallet backups must include current state for all active addresses, not just the seed.
- Addresses can be marked as "stateless-only" after backup, accepting larger signatures for those addresses.

Variable Stateful Signature Sizes. SHRINCS signatures presume variable signature size and verification complexity depending on: (1) the number of already produced stateful signatures; (2) the signing mode. In decentralized system we need to consider how the cost of the signature verification is calculated to prevent spam attacks.

- That's basically the reason this specification introduced two modes for SHRINCS. SHRINCS-B is designed for accounting systems, where the user pays for each transaction byte, so the signature size matters. SHRINCS-L is better suited to systems that calculate transaction cost based on the number of computation units consumed.
- If such or a similar post-quantum signature construction is accepted in Bitcoin, it's likely to be done with one (e.g. `OP_SHRINCS`) or the set (e.g. `OP_WOTS, OP_PORS, OP_XMSS`) of opcodes, 1 byte per each. If the signature presumes a dynamic number of hashes to be verified (and the cost of verifying "short" and "long" chains is the same), it can lead to spam with longer verification paths.
- In the case of systems where the fee additionally is calculated based on the computation complexity of the transaction, we could implement a kind of efficient pre-compiles, but in the case of dynamic signatures, we need to introduce additional parameters.

2 Notation

Throughout this specification:

`||`	denotes byte string concatenation

`[a:b]` denotes a substring from byte index `a` to `(b-1)`

`[0:a]` denotes first `a` bytes (truncation)

`←$` uniform random sampling

`n` is the security parameter in bytes

`q` is the stateful signature counter

`H` denotes a cryptographic hash function

`Tw` denotes a tweakable hash function

`PRF` denotes a pseudorandom function

$\perp$ (or `false`) used for failure returns

`|.|` denotes an array size

3 Overview

SHRINCS is designed in the way it can operate in two modes: **stateful** and **stateless**. Stateful mode is represented by the instance of Unbalanced XMSS (UXMSS) with a root public key `PK.sf`. An optimized variant of SPHINCS+ is used to support the stateless operating mode (which is represented by the public key `PK.sl`). The root public key is a hashed concatenation of (`PK.sf`, `PK.sl`), which is being converted to the Bitcoin address.

During normal operation with intact state, the user generates signatures according to efficient stateful path. Each leaf of the UXMSS is represented by WOTS+C one time signature instance. UXMSS is right-skewed, which makes the first stateful signature (`q = 1`) the cheapest from the verification and size perspective and adds 16 bytes and one additional hash operation for each next stateful signature, up to `q = hsf`. The final signature (`q = hsf + 1`) has the same size and verification cost as `q = hsf`. A right-skewed UXMSS is a tree construction in which every internal node has a one-time signature leaf as its left child and a UXMSS subtree as its right child, yielding a recursively right-branching authentication tree (as shown on the Figure 1).

If the state is corrupted (or the limit for stateful operations is met), the scheme can continue operating with stateless mode. Stateless mode utilizes an optimized SPHINCS+ variant with WOTS+C signatures in regular XMSS trees on each layer in the hypertree and PORS+FP for few-time signature scheme. The general construction of SHRINCS signature is presented on the Figure 1.

3.1 Key Pair Derivation and Size

SHRINCS keys are derived from a single random seed:

```
seed ←$ {0,1}^(3*8*n)   // 3n bytes
```

Seed allows to derive a private key `SK ← (SK.seed, SK.prf, PK.seed, PK.sf, PK.sl, PK.root)`:

```
SK.seed ← seed[0:n]     // Seed for key derivation
SK.prf ← seed[n:2n]     // Secret needed for PRF_msg
PK.seed ← seed[2n:3n]   // Domain separator for different SHRINCS key pairs
PK.sf ← compute_stateful_root(PK.seed, ADRS, SK.seed) // Public stateful key
PK.sl ← compute_stateless_root(PK.seed, ADRS, SK.seed) // Public stateless key
PK.root ← H(PK.seed, ADRS, PK.sf, PK.sl) // Root public key
```

where `ADRS` is a tweak also known as a context address (see Section 5.5).

The size of the secret key is `6n` bytes. A public key consists of two values `PK ← (PK.seed, PK.root)` and has a size of `2n` bytes.

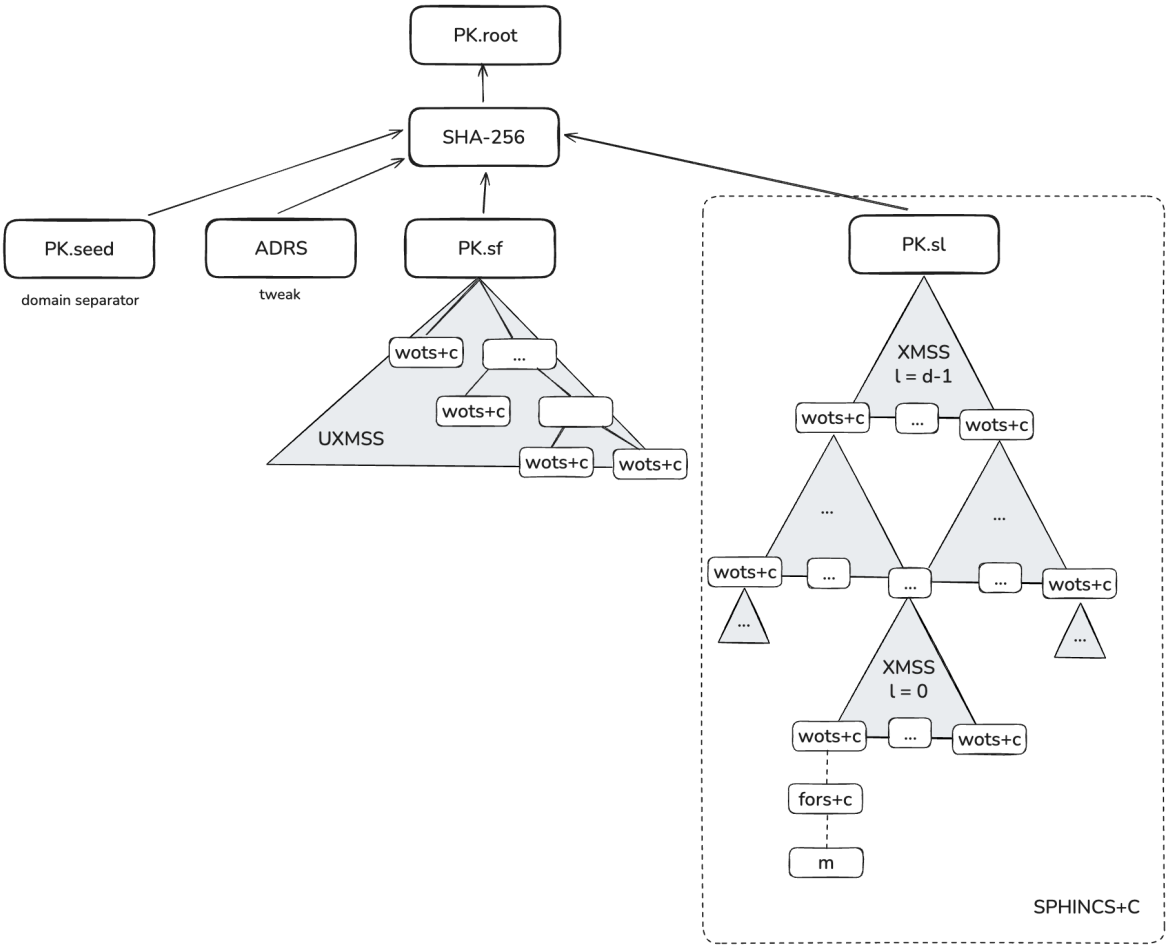

Figure 1: SHRINCS architecture

3.2 Signature Size and Verification Cost

We have different methods to calculate signature size and verification cost based on the used mode.

3.2.1 Stateful

Stateful signatures consist of:

1. `PK.sl`: the stateless public key (for unified verification)
2. `R`: randomness for message hashing (with the size `R_SIZE`)
3. `ctr`: grinding counter of the size `c`
4. `chain_values`: WOTS+C signature (`l` chains)
5. `auth_path`: UXMSS authentication path (`min(q, hsf)` nodes)

Signature size calculation:

```
General formula: size = PK.sl_SIZE + R_SIZE + c + l*n + min(q, hsf)*n
       SHRINCS-B.size = 16 + 32 + 4 + 16*16 + q*16              = 308 + 16q
       SHRINCS-L.size = 16 + 32 + 4 + 64*16 + q*16              = 1076 + 16q
```

The signature verification cost is:

```
General formula: hash_ops = (w-1)*l - S_wn + min(q, hsf) + 2
       SHRINCS-B.hash_ops = 255*16 - 2040 + q + 2              = 2042 + q
       SHRINCS-L.hash_ops = 3*64 - 140  + q + 2                = 54 + q
```

Note: `w` is the Winternitz parameter and `S_wn` is a target grinding value for WOTS+C instance (see Section 6). Two additional hashes include: (1) the hash to receive a compressed WOTS+C key; (2) the hash to obtain the root public key.

3.2.2 Stateless

Stateless signatures consist of:

1. `PK.sf`: the stateful public key (unified verification)
2. PORS+FP signature: few-time signature on the message
3. HT signature: `d` XMSS signatures forming the hypertree

Signature size calculation:

```
General formula: size = PK.sf_SIZE + pors_size(R_SIZE + (k + m_max)*n)
                  + xmss_layer_size(R_SIZE + c + l*n + h'*n) * d

SHRINCS-B.size = 16 + pors_size(32 + (6 + 91)*16)
                + xmss_layer_size(32 + 4 + 16*16 + 12*16) * 2
                = 16 + (32 + 1552) + (32 + 4 + 256 + 192) * 2
                = 16 + 1584 + 968 = 2568

SHRINCS-L.size = 16 + pors_size(32 + (6+91)*16)
                + xmss_layer_size(32 + 4 + 64*16 + 12*16) * 2
                = 16 + (32 + 1552) + (32 + 4 + 1024 + 192) * 2
                = 16 + 1584 + 2504 = 4104
```

The signature verification cost is:

```
General formula:
    pors_hashes = 2*k + m_max
    wots_per_layer = (w-1)*l - S_wn + 1
    xmss_per_layer = wots_per_layer + h'
    total = pors_hashes + d*xmss_per_layer + 1

SHRINCS-B:
    pors_hashes = 2*6 + 91 = 103
    wots_per_layer = (256-1)*16 - 2040 + 1 = 2041
    xmss_per_layer = 2041 + 12 = 2053
    total = 103 + 2*2053 + 1 = 4210 hashes

SHRINCS-L:
    pors_hashes = 103
    wots_per_layer = (w-1)*l - S_wn + 1 = 53
    xmss_per_layer = wots_per_layer + h' = 53 + 12 = 65
    total = pors_hashes + d*xmss_per_layer + 1
          = 103 + 2*65 + 1 = 234 hashes
```

Note: `k` (number of trees) and `m_max` (maximum Octopus authentication set) are parameters of PORS+FP (see Section 9). `h'` is the height of internal tree in HT. One additional hash is required to obtain the root public key after stateless public key recovery.

4 Parameters

This specification provides with the list of parameters for different SHRINCS instances, depending on verifier requirements:

1. SHRINCS-B: size matters (instance preferable for Bitcoin).
2. SHRINCS-L: verification complexity matters (the number of hashes to verify the signature).

Parameter	SHRINCS-B	SHRINCS-L	Description
`H`	`SHA-256`	`SHA-256`	Hash function
`OTS`	`WOTS+C`	`WOTS+C`	One-time signature
`FTS`	`PORS+FP`	`PORS+FP`	Few-time signature
`n`	16	16	Security parameter (bytes)
`w`	256	4	Winternitz parameter
`l`	16	64	The number of WOTS+C chains
`S_wn`	2040	140	Target sum for WOTS+C
`hsf`	141	189	Maximum stateful tree height
`hsl`	24	24	Maximum stateless hypertree height
`d`	2	2	Stateless hypertree layers
`h'`	12	12	XMSS tree height per hypertree layer `hsl/d = 12`
`t`	9245141	9245141	The number of secret values in PORS+FP tree
`b`	24	24	PORS+FP tree height
`k`	6	6	Number of PORS+FP revealed tree leafs
`m_max`	91	91	Maximum size of the Octopus authentication path
`c`	4	4	Counter size for WOTS+C grinding (bytes)
`R_SIZE`	32	32	The size (bytes) of the randomness in the signature

Note: Parameters for stateless instance is selected in the way to support up to 2^{20} signatures [KN25]. `hsf` is chosen in the way that the maximum stateful signature remains smaller than a stateless signature: `n + R_SIZE + c + l*n + hsf*n < stateless_size`.

5 Underlying Functions and Addresses

This section defines the cryptographic primitives and addressing scheme used throughout SHRINCS.

5.1 Hash Function

SHRINCS uses SHA-256 truncated to `n` bytes for all internal hash operations.

```
H(m) = SHA-256(m)[0:n]
```

SHA-256 Block Precomputation For SHA-256, the compression function `C(H,M)` operates on 64-byte blocks. Since `PK.seed` is 16 bytes, we pad it to 64 bytes to enable precomputation of the first block:

```
P = C(IV, PK.seed || 0^48)
```

Then we use the following precomputation `P` in the next compression function iteration. Every function that uses SHA-256 with it's argument involving PK.seed will use this precomputation value. This optimization allows to reduce the number of hashing operation, which leads to faster computation, especially in the grinding parts of the SHRINCS.

Additionally, we will denote the concatenation `PK.seed || 0^48` as `PK.seed*`.

5.2 Tweakable Hash Function

To prevent multi-target attacks and ensure domain separation between the various components, we employ the tweakable hash function construction defined in SPHINCS+ [Ber+19]:

```
Tw(PK.seed, ADRS, m) = H(PK.seed* || ADRS || m)
```

This usage of tweaked hashing is critical. It ensures that a hash computed at one position in the Merkle tree cannot be substituted for a hash at another position, effectively mitigating generic tree-grafting attacks.

5.3 Pseudorandom Functions

SHRINCS uses two pseudorandom functions for different purposes: key derivation and getting the randomizer for the signature.

For deterministic derivation of secret key material (WOTS+C chain starting values, PORS+FP secret keys):

```
PRF(PK.seed, ADRS, SK.seed) = Tw(PK.seed*, ADRS, SK.seed)
```

The address structure `ADRS` ensures that each derived value is unique and bound to its position in the scheme.

For generating the randomizer `R` used in message hashing:

```
PRF_msg(SK.prf, PK.seed, opt_rand, m) = HMAC-SHA-256(SK.prf, PK.seed || opt_rand ||
                                        m)[0:R_SIZE],
```

where `opt_rand` is an optional randomness (`n` bytes), can be `PK.seed` for deterministic signing. For the stateless signing path, the PORS+FP grinding counter is iterated inside `PRF_msg` to produce a fresh `R` on each attempt. The counter-augmented variant is:

```
PRF_msg_ctr(SK.prf, PK.seed, opt_rand, ctr, m) = HMAC-SHA-256(SK.prf, PK.seed ||
                            opt_rand || ctr || m)[0:R_SIZE]
```

where `ctr` is a counter incremented during PORS+FP grinding (see Section 9.4). The verifier never sees `ctr`, it only receives the final `R` that satisfied the grinding condition. This design keeps the grinding work inside the signer's PRF evaluation while allowing the verifier to check the result using only `R` and the message hash.

5.4 Message Hash Function

SHRINCS uses different hashing strategies for WOTS+C and PORS+FP, optimized for their respective use cases.

5.4.1 WOTS+C: Two-Phase Hashing

WOTS+C uses a two-phase approach for message hashing and grinding. This design is necessary because:

- User messages need randomized hashing (phase 1 + phase 2)
- Internal XMSS signatures sign tree roots that are already hashes (phase 2 only)

Phase 1. Message Digest. Computes a randomized hash of the user message:

```
H_msg(ADRS, R, PK.seed, PK.root, m) = SHA-256(PK.seed* || ADRS || R || PK.root ||
                                      m)[0:n]
```

This is computed once per signature, producing an n-byte digest.

Phase 2. Grinding Hash. Takes the digest and a counter:

```
H_grind(ADRS, PK.seed, digest, ctr) = SHA-256(PK.seed* || ADRS || digest || ctr)[0:n]
```

5.4.2 PORS+FP: Two-Phase Hashing

PORS+FP consists of two phases, where we obatain the digest, and using this digest compute the XOF output for obtaining indices in the signing process. The grinding counter is not included in the message hash. Instead, it is iterated inside `PRF_msg_ctr` (Section 5.3) to produce a fresh `R` on each grinding attempt. The verifier only sees the final `R`: For XOF we don't need to store the `blk` value either (Section 9.3).

Phase 1. Message Digest. Computes a randomized hash of the user message with the randomness `R`:

```
H_msg_pors(ADRS, R, PK.seed, PK.root, m) = SHA-256(PK.seed* || ADRS || R || PK.root ||
                                           m)[0:n]
```

The output length is n bytes.

Phase 2. XOF Block Digest. Computes a XOF hash blocks from the message digest, using `blk` to produce different blocks:

```
H_xof_pors(ADRS, R, PK.seed, PK.root, m) = SHA-256(PK.seed* || ADRS || PK.root ||
                                      digest || blk)
```

The output length of XOF is `total_bits = (roundup(2^b * k / t) + margin) * b + offset + hsl`, where `margin` is needed in the case, when the number of extracted bits is bigger than expected, `offset` is the residue bits after extracting one XOF block, and `hsl` is the number of bits needed for the hypertree leaf index extraction.

5.5 Tweak Structure and Domain Separation

Tweaks are structured as follows to ensure uniqueness across all scheme components:

```
ADRS = layer || tree_address || type || words
```

where:

`layer`: (4 bytes) layer in hypertree (`0` = bottom, `d-1` = max)

`tree_address`: (12 bytes) tree address within layer

`type`: (4 bytes) address type

`words`: (12 bytes) type-dependent 12 bytes

A total `ADRS` size is 32 bytes.

5.5.1 Address Types

```
0x00: SF_WOTS_HASH:     Stateful WOTS+C chain hashing
0x01: SF_WOTS_PK:       Stateful WOTS+C public key compression
0x02: SF_TREE:          Stateful tree internal nodes
0x03: SF_WOTS_GRIND:    Stateful WOTS+C grinding
0x04: SF_H_MSG:         Stateful message hash
0x05: SF_WOTS_PRF:      Stateful PRF for WOTS+C key derivation
0x06: PORS_HASH:        PORS+FP leaf hashing
0x07: PORS_TREE:        PORS+FP tree internal nodes
0x08: PORS_PK:          PORS+FP roots compression
```

```
0x09: PORS_PRF:          PRF for PORS+FP secret key derivation
0x0A: PORS_XOF:          XOF for PORS+FP indices extraction
0x0B: SL_WOTS_HASH:      Stateless WOTS+C chain hashing
0x0C: SL_WOTS_PK:        Stateless WOTS+C public key compression
0x0D: SL_TREE:           Stateless tree internal nodes
0x0E: SL_WOTS_GRIND:     Stateless WOTS+C grinding
0x0F: SL_H_MSG:          Stateless message hash
0x10: SL_WOTS_PRF:       Stateless PRF for WOTS+C key derivation
0x11: ROOT:              Root public key computation
```

Summarizing, `(0x00, 0x01, 0x02, 0x03, 0x04, 0x05)` are stateful only types, `(0x06, 0x07, 0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F, 0x10)` are stateless only and `(0x11)` is shared type.

5.5.2 Address Manipulation Functions

```
new_ADRS():
     return toByte(0, 32)

setLayerAddress(ADRS, layer):
    ADRS[0:4] ← toByte(layer, 4)

setTreeAddress(ADRS, tree_addr):
    ADRS[4:16] ← toByte(tree_addr, 12)

setTypeAndClear(ADRS, type):
    ADRS[16:20] ← toByte(type, 4)
    ADRS[20:32] ← toByte(0, 12)

setKeyPairAddress(ADRS, keypair):
    ADRS[20:24] ← toByte(keypair, 4)

setChainAddress(ADRS, chain):
    ADRS[24:28] ← toByte(chain, 4)

setHashAddress(ADRS, hash):
    ADRS[28:32] ← toByte(hash, 4)

setTreeHeight(ADRS, height):
    ADRS[24:28] ← toByte(height, 4)

setTreeIndex(ADRS, index):
    ADRS[28:32] ← toByte(index, 4)
```

Note that `setTreeHeight` and `setChainAddress` both write to bytes `[24:28]`. These operations are context-dependent so there mustn't be a conflict.

5.5.3 Type-Dependent Words

The list of type-dependent words for address tweaking is the following:

6 WOTS+C

The standard Winternitz One-Time Signature (WOTS+) is the building block of most modern hash-based schemes. It works by encoding the message into a series of small integers and using them to iterate hash chains [Hül13]. A significant portion of a WOTS+ signature size (approx 20%) is dedicated to "Checksum," which is necessary to prevent a forgery attack where an adversary extends the hash

Type	Name	word1 [20:24]	word2 [24:28]	word3 [28:32]
0x00	SF_WOTS_HASH	keypair	chain	hash
0x01	SF_WOTS_PK	keypair	0	0
0x02	SF_TREE	keypair	height	index
0x03	SF_WOTS_GRIND	keypair	0	0
0x04	SF_H_MSG	0	0	0
0x05	SF_WOTS_PRF	keypair	chain	0
0x06	PORS_HASH	0	0	leaf_idx
0x07	PORS_TREE	0	height	index
0x08	PORS_PK	0	0	0
0x09	PORS_PRF	tree_idx	0	leaf_idx
0x0A	PORS_XOF	0	0	0
0x0B	SL_WOTS_HASH	keypair	chain	hash
0x0C	SL_WOTS_PK	keypair	0	0
0x0D	SL_TREE	keypair	height	index
0x0E	SL_WOTS_GRIN	keypair	0	0
0x0F	SL_H_MSG	0	0	0
0x10	SL_WOTS_PRF	keypair	chain	0
0x11	ROOT	0	0	0

Table 1: Type-dependent interpretation of ADRS words `(bytes [20:32])`

chains forward. WOTS+C (compressed WOTS+) eliminates the transmission of this checksum entirely, achieving a smaller signature size.

In WOTS+, the checksum C is calculated from the message digest D.

$$C = \sum_{i=1}^{l} (w - 1 - d_i) \tag{1}$$

This checksum is then signed using additional chains. In WOTS+C [Kud+22], instead of calculating a checksum for a fixed message digest, the signer modifies the message digest until it possesses a specific, pre-determined checksum property. This is done by hashing the message with a random salt `R` (phase 1), then iterating over a counter `ctr` applied to the resulting digest (phase 2) until the output satisfies a global target condition.

6.1 The Target Condition

WOTS utilize a parameter `w` to define the number of hashchains and correspondingly their length. The message digest length is `n` bytes so `l = 8n/(log2(w))`. The values $d_{i \in [l]}$ are treated as integers in `0...(w-1)`.

Let `S_wn` be a protocol parameter (often near the expected value, but may be chosen larger to trade signing/grinding cost for verification cost). If the hash output is uniform, the expected value of a single digit is `d_i = (w-1)/2`. For `l` digits, the expected `S_wn` is:

$$S_{wn} = \sum_{i=0}^{l-1} d_i \tag{2}$$

By enforcing this condition during signing (including grinding), the verifier can simply assume the checksum is `S_wn` without receiving it. If the reconstructed digest does not sum to `S_wn`, the signature is invalid.

Additionally we can manipulate a target condition to decrease the cost of signature verification. All `l`-chains consist of `St = (w-1)*l` hashing operations. Increasing `S_wn` increases grinding cost but reduces the verification complexity to `(S_t - S_wn)` hashes. Concrete parameters are presented in Section 4.

6.2 Base-w Representation

WOTS represents a message digest in base `w` so that each digit is an integer in `[0...w-1]`. These digits directly determine the number of hashes to apply to each WOTS chain element during signing and verification. The `base_w()` function converts a byte string into an array of `out_len` base-w digits. In this specification we assume `w` is a power of two for efficiency, though this is not a strict requirement of the construction.

```
Algorithm: base_w(m, w, out_len)
    m: byte string
    w: Winternitz parameter
    out_len: output length
Output:
    b_w: array of out_len integers in [0, w-1]

1. in_idx ← 0
2. bits ← 0
3. total ← 0
4. b_w ← []
5. for i from 0 to (out_len - 1):
    a. if bits == 0:
        total ← m[in_idx]
        in_idx ← in_idx + 1
        bits ← 8
    b. bits ← bits - log2(w)
    c. b_w[i] ← (total >> bits) AND (w - 1)
6. return b_w
```

We provide optimized versions for `w = 256` and `w = 4` proposed by different SHRINCS instances. For `w=256`:

```
Algorithm: base_w256(m, out_len)
Input:
    m: byte string
    out_len: output length
Output:
    b_w: array of out_len integers in [0, 255]

1. b_w ← []
2. for i from 0 to (out_len - 1):
    a. b_w[i] ← m[i]
3. return b_w
```

For `w=4`:

```
Algorithm: base_w4(m, out_len)
Input:
    m: byte string
    out_len: output length
Output:
    b_w: array of out_len integers in [0,3]

1. b_w ← []
2. for j from 0 to (out_len/4 - 1):
    a. x ← m[j]
    b. b_w[4*j + 0] ← (x >> 6) AND 3
    c. b_w[4*j + 1] ← (x >> 4) AND 3
    d. b_w[4*j + 2] ← (x >> 2) AND 3
```

```
    e. b_w[4*j + 3] ← x AND 3
3. return b_w
```

6.3 Chain Function

WOTS derives signature components and public key material by iteratively applying the tweakable hash function along a Winternitz chain. The function `chain()` takes an n-byte input value `m` and applies `Tw(PK.seed, ADRS, ·)` exactly `steps` times, beginning at chain position `start`. For each iteration, the address field hash inside `ADRS` is set to the current chain position `i` (from `start` to `start + steps - 1`) to ensure domain separation between different chain steps. The output is the resulting n-byte value after advancing the chain by steps, which is used both for producing signature chain values (advancing from `0` to `msg[i]`) and for reconstructing end-of-chain values during verification (advancing from `msg[i]` to `w-1`).

The `mode` parameter determines which address types are used: `SF` for the stateful tree instance or `SL` for the stateless instance.

```
Algorithm: chain(m, start, steps, PK.seed, ADRS)
Input:
    m: input
    start: starting index
    steps: number of iterations
    PK.seed: public seed
    ADRS: tweak (type already set to SF_WOTS_HASH or SL_WOTS_HASH)
Output:
    n-byte chain output

1. tmp ← m
2. for i from start to (start + steps - 1):
    a. setHashAddress(ADRS, i)
    b. tmp ← Tw(PK.seed, ADRS, tmp)
3. return tmp
```

6.4 Key Generation

To create a WOTS public key for a given `keypair` index, we derive one secret starting value per chain using `PRF`, then advance each value to the end of its chain. The resulting array of end-of-chain values is compressed into a single n-byte WOTS public key via the tweakable hash. The compression uses address type `SF_WOTS_PK` (stateful) or `SL_WOTS_PK` (stateless).

```
Algorithm: wots_PKgen(SK.seed, PK.seed, ADRS, keypair, mode)
Input:
    SK.seed: secret seed
    PK.seed: public seed
    ADRS: tweak
    keypair: keypair index
    mode: SF or SL
Output:
    pk: compressed WOTS+C public key

    if mode == SF:
        WOTS_HASH       ← SF_WOTS_HASH
        WOTS_PK         ← SF_WOTS_PK
        WOTS_PRF_TYPE   ← SF_WOTS_PRF
    else:
        WOTS_HASH       ← SL_WOTS_HASH
        WOTS_PK         ← SL_WOTS_PK
        WOTS_PRF_TYPE   ← SL_WOTS_PRF
```

```
1. tmp ← []
2. for i from 0 to l - 1:
    a. setTypeAndClear(ADRS, WOTS_PRF_TYPE)
    b. setKeyPairAddress(ADRS, keypair)
    c. setChainAddress(ADRS, i)
    d. sk_i ← PRF(PK.seed, ADRS, SK.seed)
    e. setTypeAndClear(ADRS, WOTS_HASH)
    f. setKeyPairAddress(ADRS, keypair)
    g. setChainAddress(ADRS, i)
    h. tmp[i] ← chain(sk_i, 0, w - 1, PK.seed, ADRS)
3. setTypeAndClear(ADRS, WOTS_PK)
4. setKeyPairAddress(ADRS, keypair)
5. pk ← Tw(PK.seed, ADRS, tmp[0] || ... || tmp[l-1])
6. return pk
```

6.5 Message Digest Grinding

WOTS+C uses a two-phase hashing approach (see Section 5.4.1):

1. Phase 1 (`H_msg`): Computes a randomized digest of the user message. This is performed once per signature and produces an `n`-byte digest. For internal XMSS signatures (which sign tree roots that are already hashes), phase 1 is skipped and the tree root is used directly as the digest.

2. Phase 2 (`H_grind`): Takes the phase 1 digest (or tree root), `PK.seed`, and a counter, producing a candidate n-byte output. This is iterated during grinding until the target condition is met. `PK.seed` is included to bind the grinding output to the key pair.

The function `wots_grind()` takes a pre-computed digest and increments `ctr` until the base-w digits of the Phase 2 output satisfy the target sum `S_wn`. It returns the found `ctr` and the corresponding digit array.

```
Algorithm: wots_grind(PK.seed, digest, ADRS, keypair, mode)
Input:
    PK.seed: public seed
    digest: a digest from phase 1 H_msg, or a tree root for internal signatures
    ADRS: tweak
    keypair: keypair
    mode: SF or SL
Output:
    msg: array of l integers in [0, w-1]
    ctr: final counter value used

1. if mode == SF:
        setTypeAndClear(ADRS, SF_WOTS_GRIND)
    else:
        setTypeAndClear(ADRS, SL_WOTS_GRIND)
2. setKeyPairAddress(ADRS, keypair)
3. for ctr from 0 to 2^32 - 1:  // c = 4 bytes
    a. grind_out ← H_grind(ADRS, PK.seed, digest, ctr)
    b. msg ← base_w(grind_out, w, l)
    c. sum ← 0
    d. for i from 0 to (l - 1):
        sum ← sum + msg[i]
    e. if sum == S_wn:
        return (msg, ctr)
4. abort with error  // caller should resample R and retry
```

Additionally we need to define a func which allows to calculate the digest without grinding (by providing the `ctr` value directly).

```
Algorithm: wots_digest(PK.seed, digest, ctr, ADRS, keypair, mode)
Input:
    digest: digest from phase 1 H_msg, or a tree root
    ctr: counter value
    ADRS: tweak
    keypair: keypair
    mode: SF or SL
Output:
    msg: array of l integers in [0, w-1]
    valid: boolean indicating if sum == S_wn

1. if mode == SF:
        setTypeAndClear(ADRS, SF_WOTS_GRIND)
    else:
        setTypeAndClear(ADRS, SL_WOTS_GRIND)
2. setKeyPairAddress(ADRS, keypair)
3. grind_out ← H_grind(ADRS, PK.seed, digest, ctr)
4. msg ← base_w(grind_out, w, l)
5. sum ← 0
6. for i from 0 to l - 1:
    sum ← sum + msg[i]
7. valid ← (sum == S_wn)
8. return (msg, valid)
```

6.6 Signature Generation

To sign a user message, WOTS+C first computes the phase 1 digest, then grinds to find a valid counter. For each chain, it reveals the chain value after exactly `msg[i]` steps (starting from the secret-derived chain `start`). The signature carries `R`, the grinding counter `ctr`, and the `l` chain values.

For internal XMSS signatures (signing tree roots at layers `1..d-1` of the hypertree), phase 1 is skipped: the tree root serves directly as the digest input to `wots_grind`. In this case `R` is still generated and included in the signature to maintain a uniform format, but it does not affect the digest computation.

```
Algorithm: wots_sign(m, SK.seed, SK.prf, PK.seed, PK.root, ADRS, keypair, mode, is_internal)
Input:
    m: message
    SK.seed: secret seed
    SK.prf: PRF key for randomness
    PK.seed: public seed
    PK.root: public key root
    ADRS: tweak
    keypair: keypair index
    mode: SF or SL
    is_internal: boolean (true for internal XMSS tree-root signatures)
Output:
    sig: WOTS+C signature (R || ctr || chain_values)

    if mode == SF:
        WOTS_HASH     ← SF_WOTS_HASH
        WOTS_PRF_TYPE ← SF_WOTS_PRF
        H_MSG_TYPE    ← SF_H_MSG
    else:
        WOTS_HASH     ← SL_WOTS_HASH
        WOTS_PRF_TYPE ← SL_WOTS_PRF
```

```
        H_MSG_TYPE    ← SL_H_MSG

1. opt_rand ← random(n) or PK.seed     // randomized or deterministic
2. R ← PRF_msg(SK.prf, PK.seed, opt_rand, m)
3. if is_internal:
        digest ← m
    else:
        setTypeAndClear(ADRS, H_MSG_TYPE)
        digest ← H_msg(ADRS, R, PK.seed, PK.root, m)
4. (msg, ctr) ← wots_grind(PK.seed, digest, ADRS, keypair, mode)
5. sig ← R || ctr
6. for i from 0 to (l - 1):
    a. setTypeAndClear(ADRS, WOTS_PRF_TYPE)
    b. setKeyPairAddress(ADRS, keypair)
    c. setChainAddress(ADRS, i)
    d. sk_i ← PRF(PK.seed, ADRS, SK.seed)
    e. setTypeAndClear(ADRS, WOTS_HASH)
    f. setKeyPairAddress(ADRS, keypair)
    g. setChainAddress(ADRS, i)
    h. sig ← sig || chain(sk_i, 0, msg[i], PK.seed, ADRS)
7. return sig
```

6.7 Signature Verification (Public Key Recovery)

Verification reconstructs the WOTS+C public key implied by the signature. It first recomputes the message digest using the two-phase approach (or directly from the tree root for internal signatures), then verifies the target sum. For each chain element, the verifier continues hashing forward from the signature's provided intermediate value for `(w - 1 - msg[i])` steps to reach the end-of-chain value. Compressing all end-of-chain values yields the recovered WOTS+C public key.

```
Algorithm: wots_pkFromSig(sig, m, PK.seed, PK.root, ADRS, keypair, mode, is_internal)
Input:
    sig: WOTS+C signature (R || ctr || chain_values)
    m: signed message (user message or tree root)
    PK.seed: public seed
    PK.root: root public key
    ADRS: tweak
    keypair: keypair index
    mode: SF or SL
    is_internal: boolean (true for internal XMSS tree-root signatures)
Output:
    pk: recovered public key (n bytes), or "false" if invalid

    if mode == SF:
        WOTS_HASH  ← SF_WOTS_HASH
        WOTS_PK    ← SF_WOTS_PK
        H_MSG_TYPE ← SF_H_MSG
    else:
        WOTS_HASH  ← SL_WOTS_HASH
        WOTS_PK    ← SL_WOTS_PK
        H_MSG_TYPE ← SL_H_MSG

1. R ← sig[0 : R_SIZE]
2. ctr ← sig[R_SIZE : R_SIZE + c]
3. chains_start ← R_SIZE + c
4. if is_internal:
```

```
        digest ← m
    else:
        setTypeAndClear(ADRS, H_MSG_TYPE)
        digest ← H_msg(ADRS, R, PK.seed, PK.root, m)
5. (msg, valid) ← wots_digest(PK.seed, digest, ctr, ADRS, keypair, mode)
6. if not valid:
    return false
7. tmp ← []
8. for i from 0 to (l - 1):
    a. setTypeAndClear(ADRS, WOTS_HASH)
    b. setKeyPairAddress(ADRS, keypair)
    c. setChainAddress(ADRS, i)
    d. sig_i ← sig[chains_start + i * n : chains_start + (i+1) * n]
    e. tmp[i] ← chain(sig_i, msg[i], w - 1 - msg[i], PK.seed, ADRS)
9. setTypeAndClear(ADRS, WOTS_PK)
10. setKeyPairAddress(ADRS, keypair)
11. pk ← Tw(PK.seed, ADRS, tmp[0] || ... || tmp[l-1])
12. return pk
```

6.8 Contribution to the Signature Size and Verification Complexity

Each WOTS+C signature contributes the following to the enclosing signature:

- `R`: `R_SIZE` bytes
- `ctr`: `c` bytes
- chain values: `l*n` bytes

Total: `R_SIZE + c + l*n` bytes (292 bytes for SHRINCS-B and 1060 bytes for SHRINCS-L)

The verification cost is `(w-1)*l - S_wn + 1` hashes per WOTS+C instance (2041 hashes for SHRINCS-B and 53 hashes for SHRINCS-L): the verifier completes each chain forward by `(w - 1 - msg[i])` steps, and the sum of all forward steps is `(w-1)*l - S_wn` (since the sum of `msg[i]` values equals `S_wn`). One additional hash is needed for the public key compression.

7 XMSS

SHRINCS employs two distinct Merkle tree constructions: a standard balanced XMSS tree for the stateless hypertree, and an Unbalanced XMSS (UXMSS) tree optimized for the stateful component (specified in Section 8).

The stateless signature mode uses a SPHINCS+-style hypertree consisting of `d` layers, where each layer contains standard balanced XMSS trees. Each XMSS tree authenticates `2^h'` WOTS+C key pairs, where `h' = hsl / d` is the height of trees within each layer.

7.1 XMSS Tree Structure

An XMSS tree of height `h'` is a complete binary Merkle tree where:

- Leaves: The `2^h'` leaves are WOTS+C public keys, computed via `wots_PKgen` with mode `SL`.
- Internal nodes: Each internal node is the tweakable hash of its two children, using address type `SL_TREE`.
- Root: The tree root serves as either the layer's contribution to the hypertree or (for the top layer) as `PK.sl`.

Layer indexing within the stateless hypertree is the following:

- Layer 0: Bottom layer (is used to sign the PORS+FP public keys)
- Layer d-1: Top layer (root is `PK.sl`)

7.2 XMSS TreeHash

The `xmss_treehash` function computes subtree roots within an XMSS tree. It takes a target height and a leftmost leaf index, then recursively builds the subtree by computing leaves (WOTS+C public keys) at height 0 and hashing pairs of children to form internal nodes. This function is the core building block used both for computing the full tree root (by calling it with `target_height = h'` and `start_idx = 0`) and for generating individual sibling nodes needed in authentication paths (with arbitrary `start_idx`).

```
Algorithm: xmss_treehash(SK.seed, PK.seed, ADRS, target_height, start_idx)
Input:
    SK.seed: secret seed
    PK.seed: public seed
    ADRS: tweak
    target_height: height of output node (0 = leaf level)
    start_idx: leftmost leaf index in the subtree
Output:
    node: hash value

1. if target_height == 0:
    a. pk ← wots_PKgen(SK.seed, PK.seed, ADRS, start_idx, SL)
    b. return pk
2. left ← xmss_treehash(SK.seed, PK.seed, ADRS, target_height - 1, start_idx)
3. right ← xmss_treehash(SK.seed, PK.seed, ADRS, target_height - 1, start_idx +
                                                          + 2^(target_height - 1))
4. setTypeAndClear(ADRS, SL_TREE)
5. setTreeHeight(ADRS, target_height)
6. setTreeIndex(ADRS, start_idx >> target_height)
7. return Tw(PK.seed, ADRS, left || right)
```

Note: `start_idx` need not be zero. When computing a full tree root, it is called with `start_idx = 0`. When computing a sibling node for an authentication path, `start_idx` is the leftmost leaf index of that sibling's subtree.

7.3 XMSS Root Computation

The `xmss_root` function computes the root of an entire XMSS tree by invoking `xmss_treehash` with the full tree height and starting from leaf index 0. The resulting root hash represents the public key for this XMSS tree instance, which either becomes `PK.sl` (for the top hypertree layer) or is signed by the layer above (for lower layers).

```
Algorithm: xmss_root(SK.seed, PK.seed, ADRS, h_prime)
Input:
    SK.seed: secret seed
    PK.seed: public seed
    ADRS: tweak
    h_prime: tree height
Output:
    root: tree root (n bytes)

1. return xmss_treehash(SK.seed, PK.seed, ADRS, h_prime, 0)
```

7.4 XMSS Authentication Path Generation

For a leaf at index `idx`, the authentication path consists of the sibling nodes at each level from the leaf up to (but not including) the root. The authentication path contains `h_prime` nodes, each of size `n` bytes, allowing a verifier to reconstruct the path from the leaf to the root.

At height h, the sibling subtree's leftmost leaf index is determined by flipping the h-th bit of idx (using XOR) and clearing all lower bits. The sibling node is then computed by calling xmss_treehash on the subtree rooted at that position.

```
Algorithm: xmss_auth_path(SK.seed, PK.seed, ADRS, h_prime, idx)
Input:
    SK.seed, PK.seed: seeds
    ADRS: tweak
    h_prime: tree height
    idx: leaf index being authenticated
Output:
auth: authentication path (h_prime * n bytes)

1. auth ← []
2. for h from 0 to h_prime - 1:
    a. sibling_start ← (idx XOR (1 << h)) AND (0xFFFFFFFF << h)
    b. auth ← auth || xmss_treehash(SK.seed, PK.seed, ADRS, h, sibling_start)
3. return auth
```

Note: The expression (idx XOR (1 << h)) AND (0xFFFFFFFF << h) flips the h-th bit of idx to identify the sibling, then clears the lower h bits to obtain the leftmost leaf index of the sibling's subtree of height h.

7.5 XMSS Public Key Recovery from Signature

Given a WOTS+C signature and authentication path, this algorithm recovers the XMSS tree root. First, it recovers the WOTS+C public key (the leaf value) from the WOTS+C signature. Then, starting from the leaf, it iteratively combines the current node with the corresponding authentication path node to compute the parent, continuing until reaching the root. At each height h, the h-th bit of idx determines whether the current node is the left child (bit = 0) or right child (bit = 1), which dictates the ordering when hashing the pair.

The message m being verified is either a PORS+FP public key (at layer 0) or a lower-layer tree root (at layers 1...d-1). In both cases, m is already a hash, so wots_pkFromSig is called with is_internal = true (phase 1 of message hashing is skipped; see Section 6.7).

```
Algorithm: xmss_pkFromSig(wots_sig, auth, m, PK.seed, PK.root, ADRS, h_prime, idx)
Input:
    wots_sig: WOTS+C signature
    auth: authentication path (h_prime * n bytes)
    m: signed message (PORS+FP public key or lower-layer tree root)
    PK.seed: public seed
    PK.root: root public key
    ADRS: tweak
    h_prime: tree height
    idx: leaf index
Output:
    root: computed tree root (n bytes), or "false" if invalid

1. pk ← wots_pkFromSig(wots_sig, m, PK.seed, PK.root, ADRS, idx, SL, true)
2. if pk == false:
    return false
3. node ← pk
4. for h from 0 to h_prime - 1:
    a. setTypeAndClear(ADRS, SL_TREE)
    b. setTreeHeight(ADRS, h + 1)
    c. setTreeIndex(ADRS, idx >> 1)
```

```
    d. auth_node ← auth[h * n : (h + 1) * n]
    e. if (idx AND 1) == 0:
            node ← Tw(PK.seed, ADRS, node || auth_node)
        else:
            node ← Tw(PK.seed, ADRS, auth_node || node)
    f. idx ← idx >> 1
5. return node
```

7.6 XMSS Signature Generation (Stateless Layer)

To sign a message within the stateless hypertree, the XMSS signature generation produces a WOTS+C signature on the message followed by the authentication path for the corresponding leaf. The message being signed is typically either a PORS+FP public key (at layer 0) or the root of a lower-layer XMSS tree (at layers 1 through `d-1`). In both cases the message is already a hash value, so `wots_sign` is called with `is_internal = true`.

The leaf index `idx` determines which WOTS+C key pair is used; this index is determined from the message digest during stateless signing (see Section 10.5).

```
Algorithm: xmss_sign(m, SK.seed, SK.prf, PK.seed, PK.root, ADRS, h_prime, idx)
Input:
    m: message to sign (lower-layer root or PORS+FP public key)
    SK.seed: secret seed
    SK.prf: PRF key
    PK.seed: public seed
    PK.root: root public key
    ADRS: tweak
    h_prime: tree height
    idx: leaf index to use

Output:
    sig: XMSS signature (WOTS+C signature || authentication path)

1. wots_sig ← wots_sign(m, SK.seed, SK.prf, PK.seed, PK.root, ADRS, idx, SL, true)
2. auth ← xmss_auth_path(SK.seed, PK.seed, ADRS, h_prime, idx)
3. return wots_sig || auth
```

7.7 Contribution to Signature Size and Verification Complexity

Each XMSS signature consists of a WOTS+C signature plus an authentication path of `h'` nodes: `R_SIZE + c + (l + h') * n` bytes (484 bytes for SHRINCS-B and 1252 bytes for SHRINCS-L).

The full stateless hypertree consists of `d = 2` layers, giving:

- 936 bytes and 4104 hashes for SHRINCS-B
- 2472 bytes and 128 hashes for SHRINCS-L

Note: This excludes the PORS+FP contribution (see Section 9). The total stateless signature size and verification cost are the sum of PORS+FP and hypertree components, as presented in Section 3.2.2.

8 UXMSS. The Stateful Tree

Standard XMSS uses a balanced binary tree, which minimizes the maximum path length but imposes a logarithmic cost on every signature. For `N` signatures, every signature has size proportional to `log_2(N)`. SHRINCS utilizes an Unbalanced Merkle Tree (specifically a "Right-Skewed" tree) to optimize for the typical Bitcoin use case: a wallet that performs very few transactions in its lifecycle.

In this topology the q-th signature requires a path of length q. This linear growth is superior to logarithmic growth for small q. For q=1, the path is just 16 bytes. Even for q=10, the path is $10 \times 16 = 160$ bytes, which is extremely small compared to the stateless part.

8.1 UXMSS Tree Structure

The UXMSS tree is constructed recursively as a right-skewed binary tree with `hsf + 1` leaves. To describe the tree structure, we use "level" to denote depth from the root, with level 0 at the root and level `hsf - 1` at the bottom internal node. Note that this UXMSS convention is the inverse of the convention used in the balanced XMSS trees of the stateless component (Section 7). However, the ADRS height field for internal nodes is set to (`hsf - level`), so both constructions use increasing height values toward the root. The UXMSS tree is constructed as follows:

- Level 0 is the root public key `PK.sf`.
- Level `i` (for `0 <= i <= hsf-2`, internal):
  - Left child: WOTS+C public key for signature `q=i+1`
  - Right child: Root of subtree for signatures `q>i+1`
- Level `hsf-1` (bottom)
  - Left child: WOTS+C public key for signature `q=hsf`
  - Right child: WOTS+C public key for signature `q=hsf+1`

This means the tree has `hsf + 1` leaves (one per valid signature index `q` $\in \{$`1, ..., hsf + 1`$\}$), and every internal node has a WOTS+C leaf as its left child and a UXMSS subtree (or a second leaf at the bottom) as its right child.

8.2 UXMSS TreeHash

The `uxmss_treehash` function computes subtree roots within the right-skewed UXMSS tree. Unlike balanced XMSS where treehash recursively computes both subtrees symmetrically, UXMSS treehash follows the right-skewed structure: the left child is always a WOTS+C public key (a leaf), while the right child is either another UXMSS subtree (for internal levels) or a second WOTS+C public key (at the bottom level). The level parameter indicates depth from the root, with level 0 being the root and level `hsf - 1` being the bottom internal node.

All UXMSS operations use the stateful address types (`SF_WOTS_HASH`, `SF_WOTS_PK`, `SF_TREE`) and are fixed at `layer = 0`, `tree_address = 0` (since there is exactly one stateful tree per SHRINCS key pair).

```
Algorithm: uxmss_treehash(SK.seed, PK.seed, ADRS, level)
Input:
    SK.seed: secret seed
    PK.seed: public seed
    ADRS: tweak (layer = 0, tree_address = 0)
    level: depth from root (0 = root, hsf - 1 = bottom internal node)
Output:
    node: hash value (n bytes)

1. left ← wots_PKgen(SK.seed, PK.seed, ADRS, level + 1, SF)
2. if level == hsf - 1:
       right ← wots_PKgen(SK.seed, PK.seed, ADRS, hsf + 1, SF)
   else:
       right ← uxmss_treehash(SK.seed, PK.seed, ADRS, level + 1)
3. setTypeAndClear(ADRS, SF_TREE)
4. setTreeHeight(ADRS, hsf - level)
5. setTreeIndex(ADRS, 0)
6. return Tw(PK.seed, ADRS, left || right)
```

Note: The keypair index passed to `wots_PKgen` corresponds to the signature number `q`. At level `i`, the left child is the leaf for `q = i + 1`. At the bottom level (`hsf - 1`), the right child is the leaf for `q = hsf + 1`.

8.3 UXMSS Root Computation

The `uxmss_root` function computes `PK.sf`, the root of the stateful UXMSS tree. It initializes the address structure with layer 0 and tree address 0 (since there is only one stateful tree), then invokes `uxmss_treehash` starting from level 0. The resulting root, combined with `PK.sl` from the stateless tree, forms the basis for `PK.root`.

```
Algorithm: uxmss_root(SK.seed, PK.seed, ADRS)
Input:
    SK.seed: secret seed
    PK.seed: public seed
    ADRS: tweak
Output:
    PK.sf: stateful public key

1. setLayerAddress(ADRS, 0)
2. setTreeAddress(ADRS, 0)
3. return uxmss_treehash(SK.seed, PK.seed, ADRS, 0)
```

8.4 UXMSS Authentication Path Generation

For signature number `1 <= q <= hsf + 1`, the authentication path contains `min(q, hsf)` sibling nodes needed to reconstruct `PK.sf` from the WOTS+C public key at position `q`. The structure of the path differs based on whether `q <= hsf` or `q = hsf + 1`. For `q <= hsf`, the first sibling is either a subtree root (if `q < hsf`) or the rightmost WOTS+C key (if `q = hsf`), followed by q-1 WOTS+C public keys ascending toward the root. For `q = hsf + 1` (the rightmost leaf), all hsf siblings are WOTS+C public keys. This asymmetric structure reflects the right-skewed nature of UXMSS.

```
Algorithm: uxmss_auth_path(SK.seed, PK.seed, ADRS, q)
Input:
    SK.seed, PK.seed: seeds
    ADRS: address
    q: signature number (1 to hsf + 1)
Output:
    auth: authentication path (min(q, hsf)*n bytes)

1. auth ← []
2. setLayerAddress(ADRS, 0)
3. setTreeAddress(ADRS, 0)
4. if q <= hsf:
        if q == hsf:
            auth ← auth || wots_PKgen(SK.seed, PK.seed, ADRS, hsf + 1, SF)
        else:
            auth ← auth || uxmss_treehash(SK.seed, PK.seed, ADRS, q)
        for i from 1 to q - 1:
            auth ← auth || wots_PKgen(SK.seed, PK.seed, ADRS, q - i, SF)
5. else:
        for i from 0 to hsf - 1:
            auth ← auth || wots_PKgen(SK.seed, PK.seed, ADRS, hsf - i, SF)
6. return auth
```

Note: For `q <= hsf`, the authentication path has exactly `q` nodes. For `q = hsf + 1`, it has `hsf` nodes, since the rightmost leaf shares its parent's other child (a WOTS+C key) rather than a subtree root. Consequently, both `q = hsf` and `q = hsf + 1` produce authentication paths of identical byte length (`hsf * n` bytes). A verifier receiving only the signature cannot determine `q` from the authentication path length alone when the path has `hsf` nodes. This ambiguity is resolved during verification by trying both candidate values and accepting whichever reconstructs a valid `PK.sf` root.

8.5 UXMSS Public Key Recovery from Signature

Given a WOTS+C signature and authentication path, this algorithm reconstructs `PK.sf`. First, it recovers the WOTS+C public key from the signature. Then, starting from this leaf, it iteratively combines the current node with authentication path nodes to compute parent nodes until reaching the root.

The combination order follows from the tree structure:

- Case `q <= hsf`: The recovered key is a left child. The first combination pairs the recovered key (left) with the first auth node (right). All subsequent combinations have the auth node (a WOTS+C key from a lower `q` value) on the left and the current node on the right.
- Case `q = hsf + 1`: The recovered key is the rightmost leaf (a right child). All auth nodes are WOTS+C keys on the left.

```
Algorithm: uxmss_pkFromSig(wots_sig, auth, m, PK.seed, PK.root, ADRS, q)
Input:
    wots_sig: WOTS+C signature
    auth: authentication path
    m: signed message
    PK.seed: public seed
    PK.root: root public key
    ADRS: address
    q: signature number
Output:
    root: computed PK.sf, or "false" if invalid

1. setLayerAddress(ADRS, 0)
2. setTreeAddress(ADRS, 0)
3. pk ← wots_pkFromSig(wots_sig, m, PK.seed, PK.root, ADRS, q, SF, false)
4. if pk == false:
    return false
5. node ← pk
6. setTypeAndClear(ADRS, SF_TREE)
7. if q <= hsf:
    a. setTreeHeight(ADRS, hsf - (q - 1))
    b. setTreeIndex(ADRS, 0)
    c. node ← Tw(PK.seed, ADRS, node || auth[0 : n])
    for i from 1 to q - 1:
        a. setTreeHeight(ADRS, hsf - (q - 1 - i))
        b. setTreeIndex(ADRS, 0)
        c. node ← Tw(PK.seed, ADRS, auth[i * n : (i + 1) * n] || node)
8. else:
    for i from 0 to hsf - 1:
        a. setTreeHeight(ADRS, i + 1)
        b. setTreeIndex(ADRS, 0)
        c. node ← Tw(PK.seed, ADRS, auth[i * n : (i + 1) * n] || node)
9. return node
```

Note: In step 3, `is_internal = false` because the stateful UXMSS signs user messages directly (requiring phase 1 + phase 2 hashing). This differs from the stateless XMSS layers which always sign tree roots or PORS+FP public key (`is_internal = true`).

8.6 UXMSS Signature Generation

To sign a message using the stateful path, UXMSS signature generation produces a WOTS+C signature followed by the authentication path for position `q`. The signature number `q` must be tracked as state and incremented after each use to ensure one-time signature security. As noted in Section 1.2, the state counter should be persisted before generating the signature to prevent key reuse in case of interruption.

The resulting signature size is `|wots_sig| + min(q, hsf)*n` bytes, which grows linearly with `q` up to `q = hsf`. The last two signatures (`q = hsf` and `q = hsf + 1`) have identical size. For the first signature (`q = 1`), this yields the minimal signature that makes SHRINCS attractive for Bitcoin applications.

```
Algorithm: uxmss_sign(m, SK.seed, SK.prf, PK.seed, PK.root, ADRS, q)
Input:
    m: message
    SK.seed: secret seed
    SK.prf: PRF key
    PK.seed: public seed
    PK.root: root public key
    ADRS: address
    q: signature number
Output:
    sig: UXMSS signature (WOTS+C signature || authentication path)

1. setLayerAddress(ADRS, 0)
2. setTreeAddress(ADRS, 0)
3. wots_sig ← wots_sign(m, SK.seed, SK.prf, PK.seed, PK.root, ADRS, q, SF, false)
4. auth ← uxmss_auth_path(SK.seed, PK.seed, ADRS, q)
5. return wots_sig || auth
```

8.7 Contribution to Signature Size and Verification Complexity

Each UXMSS signature consists of a WOTS+C signature plus an authentication path whose length depends on the signature index `q`:

```
total = R_SIZE + c + l*n + min(q, hsf)*n
```

For concrete instances and signatures' number:

```
SHRINCS-B:
    q=1:         32 + 4 + 16*16 + 16 =       308 bytes
    q=10:        32 + 4 + 16*16 + 16*10 =    452 bytes
    q=158:       32 + 4 + 16*16 + 16*158 =  2820 bytes
    q=159(max): 32 + 4 + 16*16 + 16*158 =  2820 bytes // same as q=158; < 2856 (stateless)
SHRINCS-L:
    q=1:         32 + 4 + 64*16 + 16 =       1076 bytes
    q=10:        32 + 4 + 64*16 + 16*10 =    1220 bytes
    q=206:       32 + 4 + 64*16 + 16*206 =   4356 bytes
    q=207(max): 32 + 4 + 64*16 + 16*206 =   4356 bytes // same as q=206; < 4392 (stateless)
```

Verification cost per UXMSS signature is calculated as WOTS+C verification cost + `min(q, hsf)` hashes.

9 PORS+FP

PORS (PRNG to Obtain a Random Subset) is a few-time signature scheme that uses a *single* Merkle tree over `t` leaves. PORS reveals `k` leaves from a single tree and uses the Octopus algorithm [AE18] to compute the minimal set of authentication nodes needed to verify all `k` leaves simultaneously, exploiting shared nodes to shorten the proof. PORS+FP [AK25] adds *forced pruning* on top: the signer grinds a counter until the resulting k-subset of leaves produces an Octopus authentication set of size at most `m_max` nodes, bounding the signature to a fixed maximum size.

9.1 PORS Structure

PORS consists of a single Merkle tree over `t` leaves:

- The tree has `t` leaves (not necessarily a power of 2; the tree is left-filled as described in Section 2.1)
- Each leaf is derived from `SK.seed` using `PRF` with address type `PORS_PRF`
- The message together with a grinding randomness `R`, is hashed to a n-byte digest which is then expanded into `k` distinct indices in $[t] = \{0, 1, ..., t-1\}$
- The Octopus algorithm computes the minimal authentication set `A` for those `k` leaves

The PORS+FP public key is the tweakable hash of the single tree root:

```
PORS+FP PK = Tw(PK.seed, ADRS, root),
```

using address type `PORS_PK`. Forced pruning guarantees $|A| \leq m_{\max}$, bounding the signature to a fixed maximum size.

9.2 Grinding Condition

PORS+FP uses the same two-call message-hash interface (Section 5.4.2):

```
R      ← PRF_msg_ctr(SK.prf, PK.seed, opt_rand, ctr, m)
digest ← H_msg_pors(ADRS, R, PK.seed, PK.root, m)
A      ← pors_octopus(pors_digest_to_indices(digest))
condition: |A| <= m_max
```

The signer searches for an `R` that satisfies this condition by iterating a counter inside `PRF_msg_ctr`. Each counter value produces a fresh `R`, which is then evaluated via `H_msg_pors`. The verifier only sees the final `R` that satisfied the condition and recomputes the same digest deterministically. Expected grinding cost depends on the distribution of $|A|$;

9.3 Digest to Indices

The `pors_digest_to_indices` function converts a message digest and a grinding counter into `k` *distinct* indices in $\{0, 1, ..., t-1\}$. Distinctness is required by the Octopus algorithm. We use rejection sampling over SHA256 XOF output.

```
Algorithm: pors_digest_to_indices(digest)
Input:
    digest: 32-byte value
Output:
    indices: array of k distinct integers in [0, t-1]
    xof_block: XOF output with size 256*blk

1. b       ← roundup(log2(t))         // bits per candidate
2. c       ← floor(256 / b)        // number of candidates per 256-bit block
3. min_blk ← roundup(roundup(2^b * k / t)/c) // expected number of XOF blocks
4. indices ← []
```

```
5. xof_out ← ""
6. setTypeAndClear(ADRS, PORS_XOF)
7. for blk from 0 to 2^32 - 1:
    a. block ← H_xof_pors(ADRS, PKseed, digest, blk)
    b. xof_out ← xof_out || block
    c. for i from 0 to c - 1:
        if |indices| == k:
            break
        candidate ← extract_bits(block, i * b, b)
        if candidate < t and candidate not in indices:
            indices ← indices || [candidate]
    d. if |indices| == k and blk = min_blk - 1:
        return (indices, xof_out[0:total_bits])
8. abort with error
```

where `total_bits = (roundup(2^b * k / t) + margin) * b + offset + hsl` is the length of the XOF output (see Section 5.4.2). In our case, for chosen parameters of SHRINCS-B and SHRINCS-L, and `margin = 7`, `offset = 16`, we get the follwing length: `total_bits = (roundup(2^24 * 6 / 9245141)) * 24 + 16 + 24 = 472` bits or 59 bytes.

Note: `extract_bits(x, offset, length)` extracts length bits from byte string `x` starting at bit position offset, returned as an unsigned integer. Bits are numbered in big-endian order: bit `0` is the most significant bit of byte `x[0]`.

9.4 Octopus for the PORS+FP Tree

The Octopus algorithm [AE18] computes the minimal set of authentication nodes whose values are sufficient to recompute the tree root from `k` revealed leaves. It is invoked during grinding (to check the size condition), during signing (to determine which node values to include in the signature), and implicitly during verification (to determine which nodes to read from the signature). Because all three parties run the same algorithm on the same index set, the authentication node ordering is unambiguous.

For an imperfect, left-filled tree of `t` leaves we initialise the algorithm by partitioning the `k` input indices by their actual depth in the tree, then proceed identically to the standard Octopus formulation.

```
Algorithm: pors_octopus(indices)
Input:
    indices: array of k distinct leaf indices in [0, t-1], sorted ascending
Output:
    A:       array of authentication node (lvl, idx) pairs

1. h ← ceil(log2(t))
2. s ← t - 2^(h-1)
3. I ← []
4. P ← []
5. A ← []
6. for each i in indices:
    a. if i < 2 * s:
        I ← I || [(0, i)]
    b. else:
        P ← P || [(1, i - s)]
7. for current_lvl from 0 to h - 1:
    a. next_P ← []
    b. i ← 0
    c. while i < |I|:
        (lvl, idx) ← I[i]
        sib_idx ← idx XOR 1
```

```
            if i + 1 < |I| and I[i+1][1] == sib_idx:
                i ← i + 2
            else:
                A ← A || [(current_lvl, sib_idx)]
                i ← i + 1
            next_P ← next_P || [(current_lvl + 1, idx >> 1)]

        d. I ← next_P || P
        e. P ← []
        f. if |I| == 1 and I[0][1] == 0:
            break
    8. return A
```

Note: When `t` is a power of 2, $s = t/2$ and all leaves are at depth `h`, reducing step 4 to $I = \{(h, i) : i \in$ `indices`$\}$ and recovering the standard Octopus formulation. The algorithm performs no hash evaluations; its cost is $O(k \log t)$ set operations.

9.5 Grinding

The `pors_grind` function searches for a randomness `R` such that the Octopus authentication set for the resulting index set has size at most `m_max`. The winning `R` is returned and included directly in the enclosing SPHINCS signature; the verifier recomputes `digest` and `indices` deterministically from `R` and `m`.

```
Algorithm: pors_grind(m, SK.prf, PK.seed, PK.root, ADRS, opt_rand)
Input:
    m:       message
    SK.prf:  PRF key for randomness generation
    PK.seed: public seed
    PK.root: public key root
    ADRS:    tweak
    opt_rand: optional randomness (n bytes)
Output:
    indices: array of k integers in [0, t-1]
    R:       randomness satisfying the forced-pruning condition
    digest:  H_msg_pors output for that R

1. setTypeAndClear(ADRS, SL_H_MSG)
2. for ctr from 0 to 2^32 - 1:
    a. R      ← PRF_msg_ctr(SK.prf, PK.seed, opt_rand, ctr, m)
    b. digest ← H_msg_pors(ADRS, R, PK.seed, PK.root, m)
    c. indices ← pors_digest_to_indices(digest)
    d. A       ← pors_octopus(indices)
    e. if |A| <= m_max:
           return (indices, R, digest)
3. abort with error
```

Note: Each iteration costs one `PRF_msg_ctr` evaluation plus one `H_msg_pors` evaluation and dominates the Octopus authentication cost. The address type `SL_H_MSG` is set at the start and remains fixed for the duration of grinding.

9.6 Secret Key Generation

The `pors_SKgen` function derives the secret value for a single PORS leaf.

```
Algorithm: pors_SKgen(SK.seed, PK.seed, ADRS, leaf_idx)
```

```
Input:
    SK.seed:  secret seed
    PK.seed:  public seed
    ADRS:     tweak
    leaf_idx: leaf index in [0, t-1]
Output:
    sk: secret value (n bytes)

1. setTypeAndClear(ADRS, PORS_PRF)
2. setKeyPairAddress(ADRS, 0)
3. setTreeIndex(ADRS, leaf_idx)
4. sk ← PRF(PK.seed, ADRS, SK.seed)
5. return sk
```

9.7 TreeHash

The `pors_treehash` function recursively computes the value associated with node `(target_height, idx)` in the single PORS+FP Merkle tree, where `target_height` is the height of the node counted upward from the leaves and `idx` is its horizontal index at that height. The tree is left-filled with `t` leaves: letting $h = \lceil \log t \rceil$ and $s = t - 2^{h-1}$, the leftmost $2s$ leaves sit at depth h (or height 0) and the remaining $t - 2s$ leaves sit at depth $h - 1$ (height 1).

```
Algorithm: pors_treehash(SK.seed, PK.seed, ADRS, target_height, idx)
Input:
    SK.seed, PK.seed: seeds
    ADRS:             tweak
    target_height:    height of output node
    idx:        horizontal index of the node at target_height
Output:
    node: hash value (n bytes)

1. h ← ceil(log2(t))
2. s ← t - 2^(h-1)
3. if target_height == 0:
    a. sk ← pors_SKgen(SK.seed, PK.seed, ADRS, idx)
    b. setTypeAndClear(ADRS, PORS_HASH)
    c. setKeyPairAddress(ADRS, 0)
    d. setTreeHeight(ADRS, 0)
    e. setTreeIndex(ADRS, idx)
    f. return Tw(PK.seed, ADRS, sk)
4. if target_height == 1 and idx >= s:
    a. leaf_idx ← s + idx
    b. sk ← pors_SKgen(SK.seed, PK.seed, ADRS, idx)
    c. setTypeAndClear(ADRS, PORS_HASH)
    d. setKeyPairAddress(ADRS, 0)
    e. setTreeHeight(ADRS, 0)
    f. setTreeIndex(ADRS, idx)
    g. return Tw(PK.seed, ADRS, sk)
5. left  ← pors_treehash_paper(SK.seed, PK.seed, ADRS, target_height - 1, 2*idx)
6. right ← pors_treehash_paper(SK.seed, PK.seed, ADRS, target_height - 1, 2*idx + 1)
7. setTypeAndClear(ADRS, PORS_TREE)
8. setKeyPairAddress(ADRS, 0)
9. setTreeHeight(ADRS, target_height)
10. setTreeIndex(ADRS, idx)
11. return Tw(PK.seed, ADRS, left || right)
```

9.8 PORS+FP Authentication path

The `pors_auth_path` function generates the minimal set of authentication nodes required to verify all `k` revealed leaves simultaneously within the single PORS+FP Merkle tree. Rather than authenticating each leaf independently, it employs the Octopus algorithm to identify shared internal nodes, which significantly shortens the resulting proof. For every node coordinate `(lvl, idx)` returned by the Octopus algorithm, the function computes the required hash value by invoking `pors_treehash`. In this context, `lvl` specifies the height of the authentication node, and `idx` represents the leftmost leaf index of the subtree rooted at that node.

```
Algorithm: pors_auth_path(SK.seed, PK.seed, ADRS, indices)
    Input:
        SK.seed, PK.seed: seeds
        ADRS:    tweak
        indices: array of k distinct leaf indices in [0, t-1], sorted ascending
    Output:
        auth:    authentication path (sequence of n-byte hashes)

    1. auth ← []
    2. A ← pors_octopus(indices)
    3. for each (lvl, idx) in A:
        a. node ← pors_treehash(SK.seed, PK.seed, ADRS, lvl, idx)
        b. auth ← auth || node
    4. return auth
```

9.9 Signature generation

The `pors_sign` function produces a PORS+FP signature. Due to forced pruning, the Octopus authentication set is guaranteed to contain at most $m_{\max}$ nodes. The signature format is:

`R || sk_{i_1} || ... || sk_{i_k} || y_{a_1} || ... || y_{a_|A|}`

where $\{i_1, \ldots, i_k\} = I$ are the `k` revealed leaf indices and $\{a_1, \ldots, a_{|A|}\} = A$ are the Octopus authentication node labels (in the canonical Octopus traversal order). We pad additional n-byte 0 strings to make the length of the signature be fixed with exactly size of `(k + m_max) * n` bytes.

```
Algorithm: pors_sign(m, SK.seed, SK.prf, PK.seed, PK.root, ADRS)
Input:
    m: message
    SK.seed: secret seed
    SK.prf: PRF key
    PK.seed: public seed
    PK.root: public key root
    ADRS: tweak
Output:
    sig: PORS+FP signature
    digest: the digest that satisfied the grinding condition

1. opt_rand ← random(n) or PK.seed
2. (indices, R, digest) ← pors_grind(m, SK.prf, PK.seed, PK.root, ADRS, opt_rand)
3. sig ← R
4. for each i in indices (in increasing order):
    a. sk_i ← pors_SKgen(SK.seed, PK.seed, ADRS, i)
    b. sig  ← sig || sk_i
5. auth_path ← pors_auth_path(SK.seed, PK.seed, ADRS, indices)
6. sig ← sig || auth_path
7. for ctr from |auth_path| to m_max - 1:
    a. sig ← sig || toByte(0, n)
8. return (sig, digest)
```

9.10 Public Key Recovery

The `pors_pkFromSig` function recovers the PORS+FP public key from a signature. For each of the `k` revealed leaves, it hashes the secret key to obtain the leaf value, then runs the Octopus-guided root recomputation using the supplied authentication nodes. Verification succeeds if and only if the recovered root matches `PK.root`. We also check the correctness of padding in the signature. The function returns `"false"` if the grinding condition is not satisfied (i.e., the Octopus authentication set exceeds `m_max`) or if the padding values are not all 0^n.

```
Algorithm: pors_pkFromSig(sig, indices, PK.seed, PK.root, ADRS)
 Input:
     sig:     PORS+FP signature (R || secret leaves || auth nodes)
     indices: PORS+FP k indices from digest
     PK.seed: public seed
     PK.root: public key root
     ADRS:    tweak (layer and tree_address set by caller)
 Output:
     pk: recovered PORS public key (n bytes)

 1. offset ← R_SIZE
 2. h ← ceil(log2(t))
 3. s ← t - 2^(h-1)
 4. I ← []
 5. P ← []
 6. A ← pors_octopus(indices)
 7.  if |A| > m_max:
         return false
 8. for i from 0 to k - 1:
         a. sk_i ← sig[offset : offset + n]
         b. offset ← offset + n
         c. setTypeAndClear(ADRS, PORS_HASH)
         d. setKeyPairAddress(ADRS, 0)
         e. setTreeHeight(ADRS, 0)
         f. setTreeIndex(ADRS, indices[i])
         g. val ← Tw(PK.seed, ADRS, sk_i)
         h. if indices[i] < 2*s:
             I ← I || [(0, indices[i],     val)]
         i. else:
             P ← P || [(1, indices[i] - s, val)]
 9. for current_lvl from 0 to h - 1:
     a. paired ← [false] * |I|
     b. for i from 0 to |I| - 1:
         if i + 1 < |I| and I[i+1][1] == I[i][1] XOR 1:
             paired[i]     ← true
             paired[i + 1] ← true
     c. next_P ← []
     d. for i from 0 to |I| - 1:
         (lvl, idx, val) ← I[i]
         if paired[i] and (idx AND 1) == 0:
             continue
         setTypeAndClear(ADRS, PORS_TREE)
         setKeyPairAddress(ADRS, 0)
         setTreeHeight(ADRS, current_lvl + 1)
         setTreeIndex(ADRS, idx >> 1)
         if paired[i]:
             sib_val ← I[i-1][2]
             parent_val ← Tw(PK.seed, ADRS, sib_val || val)
```

```
            else:
                auth_val ← sig[offset : offset + n]
                offset ← offset + n
                if (idx AND 1) == 0:
                    parent_val ← Tw(PK.seed, ADRS, val || auth_val)
                else:
                    parent_val ← Tw(PK.seed, ADRS, auth_val || val)

            next_P ← next_P || [(current_lvl + 1, idx >> 1, parent_val)]

        e. I ← next_P || P
        f. P ← []
    10. for i from |A| to m_max - 1:
        a. pad_val ← sig[offset : offset + n]
        b. if pad_val != toByte(0, n):
               return false
        c. offset ← offset + n
    11. root ← I[0][2]
    12. setTypeAndClear(ADRS, PORS_PK)
    13. pk ← Tw(PK.seed, ADRS, root)
    14. return pk
```

10 SHRINCS

This section specifies the complete SHRINCS signature scheme by composing the building blocks defined in previous sections.

10.1 Key Generation

The `SHRINCS.KeyGen` function generates a fresh key pair and initializes the signing state. It samples a 3n-byte random seed, partitions it into `SK.seed` (for key derivation), `SK.prf` (for message randomization), and `PK.seed` (for domain separation). It then computes `PK.sf` (the stateful UXMSS root) and `PK.sl` (the stateless hypertree root), combining them into `PK.root` via a tweakable hash. The initial state sets `q=0` with `valid=true`, indicating the stateful path is available starting from signature number 1.

```
Algorithm: SHRINCS.KeyGen()
Output:
    SK: secret key
    PK: public key
    state: initial signing state

1. seed ←$ {0,1}^(3n)
2. SK.seed ← seed[0:n]
3. SK.prf ← seed[n:2n]
4. PK.seed ← seed[2n:3n]
5. ADRS ← new_ADRS()
6. PK.sf ← uxmss_root(SK.seed, PK.seed, ADRS)
7. setLayerAddress(ADRS, d - 1)
8. setTreeAddress(ADRS, 0)
9.  PK.sl ← xmss_root(SK.seed, PK.seed, ADRS, h_prime)
10. setLayerAddress(ADRS, 0)
11. setTreeAddress(ADRS, 0)
12. setTypeAndClear(ADRS, ROOT)
13. PK.root ← Tw(PK.seed, ADRS, PK.sf || PK.sl)
14. SK ← (SK.seed, SK.prf, PK.seed, PK.sf, PK.sl, PK.root)
15. PK ← (PK.seed, PK.root)
```

```
16. state ← { q: 0, valid: true }
17. return (SK, PK, state)
```

10.2 Recovery from Seed

The `SHRINCS.Restore` function recovers a key pair from a backed-up seed. This enables the standard Bitcoin workflow of restoring a wallet from a seed phrase. However, since the signing state (which `q` values have been used) cannot be recovered from the seed alone, the stateful path is disabled by setting `valid=false`. The recovered wallet can only use stateless signatures, which are larger but do not require state tracking. This is the key tradeoff that enables static backups while maintaining security.

```
Algorithm: SHRINCS.Restore(seed)
Input:
    seed: master seed
Output:
    SK: secret key
    PK: public key
    state: invalid state (stateless only)

1. SK.seed ← seed[0:n]
2. SK.prf ← seed[n:2n]
3. PK.seed ← seed[2n:3n]
4. ADRS ← new_ADRS()
5. PK.sf ← uxmss_root(SK.seed, PK.seed, ADRS)
6. setLayerAddress(ADRS, d - 1)
7. setTreeAddress(ADRS, 0)
8.  PK.sl ← xmss_root(SK.seed, PK.seed, ADRS, h_prime)
9.  setLayerAddress(ADRS, 0)
10. setTreeAddress(ADRS, 0)
11. setTypeAndClear(ADRS, ROOT)
12. PK.root ← Tw(PK.seed, ADRS, PK.sf || PK.sl)
13. SK ← (SK.seed, SK.prf, PK.seed, PK.sf, PK.sl, PK.root)
14. PK ← (PK.seed, PK.root)
15. state ← { q: false, valid: false }
16. return (SK, PK, state)
```

10.3 Stateful signing

The `SHRINCS.SignStateful` function signs a message using the efficient stateful (UXMSS) path. It first checks that state is valid and that signature slots remain (`q <= hsf + 1`). It then increments `q` and updates the state before generating the signature. This ordering is critical: if the process is interrupted after signing but before state persistence, the one-time key would be consumed without the state reflecting it, risking catastrophic key reuse on retry (see Section 1.2).

The signature includes `PK.sl` prepended to enable unified verification. This produces minimal signatures (324 bytes for q = 1 in SHRINCS-B) that grow linearly with `q`. If state is invalid or exhausted, the function returns "false" and the caller should use stateless signing instead.

```
Algorithm: SHRINCS.SignStateful(m, SK, state)
Input:
    m: message to sign
    SK: secret key (SK.seed, SK.prf, PK.seed, PK.sf, PK.sl, PK.root)
    state: current signing state { q, valid }
Output:
    sig: stateful signature, or $\bot$ if state is invalid/exhausted
    state_prime: updated state

1. if not state.valid:
```

```
    return (false, state)
2. q ← state.q + 1
3. if q > hsf + 1:
    return (false, state)
4. state_prime ← { q: q, valid: true }
5. ADRS ← new_ADRS()
6. uxmss_sig ← uxmss_sign(m, SK.seed, SK.prf, PK.seed, PK.root, ADRS, q)
7. sig ← PK.sl || uxmss_sig
8. return (sig, state_prime)
```

10.4 Stateless signing

The `SHRINCS.SignStateless` function signs a message using the stateless (SPHINCS+C) path. This is used when state is lost, corrupted, or exhausted.

SHRINCS uses a unified digest for the stateless path: a single call to `pors_digest_to_indices` produces a XOF output that encodes both the PORS+FP leaf indices (determining which leaves to reveal in the few-time signature) and the hypertree tree/leaf indices (determining which XMSS trees and WOTS+C key pairs to use at each layer). The grinding counter `ctr` is iterated until the grinding condition is satisfied on this unified digest.

Let `C := roundup(k * 2^b / t) + 7`, so the XOF output length is `roundup((C * b + 16 + hsl)/8)`. The first `C * b + 16` bits encode the `k` PORS+FP leaf indices. The remaining `hsl` bits encode the hypertree path indices.

The signature is constructed bottom-up: first a PORS+FP signature on the message, then `d` XMSS signatures on successive layer roots ascending through the hypertree. The signature includes `PK.sf` prepended to enable unified verification.

```
Algorithm: SHRINCS.SignStateless(m, SK)
Input:
    m: message to sign
    SK: secret key
Output:
    sig: stateless signature

1.  ADRS ← new_ADRS()
2.  setLayerAddress(ADRS, 0)
3.  setTreeAddress(ADRS, 0)
4.  (pors_sig, digest) ← pors_sign(m, SK.seed, SK.prf, PK.seed, PK.root, ADRS)
5.  indices, xof_out ← pors_digest_to_indices(digest)
6.  (tree_idx, leaf_idx) ← parse_idx(xof_out)
7.  setLayerAddress(ADRS, 0)
8.  setTreeAddress(ADRS, tree_idx[0] * 2^h_prime + leaf_idx[0])
9.  pors_pk ← pors_pkFromSig(pors_sig, indices, PK.seed, ADRS)
10. ht_sig ← []
11. msg ← pors_pk
12. for layer from 0 to d - 1:
    a. setLayerAddress(ADRS, layer)
    b. setTreeAddress(ADRS, tree_idx[layer])
    c. xmss_sig ← xmss_sign(msg, SK.seed, SK.prf, PK.seed, PK.root, ADRS, h', leaf_idx[layer])
    d. ht_sig ← ht_sig || xmss_sig
    e. if layer < d - 1:
           msg ← xmss_root(SK.seed, PK.seed, ADRS, h') // we may cache or reuse the root compute
13. sig ← PK.sf || pors_sig || ht_sig
14. return sig
```

10.5 Index Parsing

The parse_idx function extracts tree and leaf indices for each hypertree layer from the stateless digest. In the unified digest scheme, these bits follow the XOF output from pors_digest_to_indices, resulting in C * b + 16 bits used for PORS+FP indices. The hsl bits are partitioned into d pairs of (leaf_idx, tree_idx), extracted from least significant (layer 0) to most significant (layer d-1), with h' bits per leaf index.

```
Algorithm: parse_idx(xof_out)
Input:
    xof_out: XOF output
Output:
    tree_idx: array of d tree indices
    leaf_idx: array of d leaf indices

1. ht_offset ← roundup(2^b * k/t + 7) * b + 16
2. ht_bits ← extract_bits(xof_out, ht_offset, hsl)
3. idx ← ht_bits
4. tree_idx ← []
5. leaf_idx ← []
6. for layer from 0 to d - 1:
    a. leaf_idx[layer] ← idx AND (2^h_prime - 1)
    b. idx ← idx >> h_prime
    c. tree_idx[layer] ← idx
7. return (tree_idx, leaf_idx)
```

Note: The total XOF output must encode C * b + hsl = 18 * 24 + 16 + 24 = 472 bits.

10.6 Stateful verification

The SHRINCS.VerifyStateful function verifies a stateful signature. It extracts PK.sl from the signature, parses the UXMSS signature, and infers q from the authentication path length. It then uses the two-phase WOTS+C hashing to recover PK.sf via uxmss_pkFromSig and verifies that Tw(PK.seed, ADRS, PK.sf || PK.sl) equals PK.root.

```
Algorithm: SHRINCS.VerifyStateful(m, sig, PK)
Input:
    m: message
    sig: stateful signature
    PK: public key
Output:
    valid: boolean

1. ADRS ← new_ADRS()
2. PK.sl ← sig[0 : n]
3. uxmss_sig ← sig[n : ]
4. wots_sig_size ← R_SIZE + c + l * n
5. auth_len ← len(uxmss_sig) - wots_sig_size
6. if auth_len < 0 or (auth_len mod n) != 0:
    return false
7. q_raw ← auth_len / n
8. if q_raw < 1 or q_raw > hsf:
    return false
9. wots_sig ← uxmss_sig[0 : wots_sig_size]
10. auth ← uxmss_sig[wots_sig_size : ]
11. if q_raw < hsf:
        candidates ← {q_raw}
```

```
    else:
        candidates ← {hsf, hsf + 1}
12. for each q in candidates:
    a. ADRS ← new_ADRS()
    b. PK.sf ← uxmss_pkFromSig(wots_sig, auth, m, PK.seed, PK.root, ADRS, q)
    c. if PK.sf == false:
        continue
    d. setLayerAddress(ADRS, 0)
        setTreeAddress(ADRS, 0)
        setTypeAndClear(ADRS, ROOT)
        expected_root ← Tw(PK.seed, ADRS, PK.sf || PK.sl)
    e. if expected_root == PK.root:
        return true
13. return false
```

10.7 Stateless verification

The `SHRINCS.VerifyStateless` function verifies a stateless signature. It extracts `PK.sf` from the signature, recomputes the unified digest from the PORS+FP signature's `R` and `ctr`, extracts both PORS+FP indices and hypertree indices from the same digest, then verifies bottom-up through the hypertree.

```
Algorithm: SHRINCS.VerifyStateless(m, sig, PK)
Input:
    m: message
    sig: stateless signature
    PK: public key
Output:
    valid: boolean

1.  ADRS ← new_ADRS()
2.  PK.sf ← sig[0 : n]
3.  offset ← n
4.  pors_sig_size ← R_SIZE + (k + m_max)*n
5.  pors_sig ← sig[offset : offset + pors_sig_size]
6.  offset ← offset + pors_sig_size
7.  R ← pors_sig[0 : R_SIZE]
8.  setTypeAndClear(ADRS, SL_H_MSG)
9.  digest ← H_msg_pors(ADRS, R, PK.seed, PK.root, m)
10. indices, xof_out ← pors_digest_to_indices(digest)
11. A ← pors_octopus(indices)
11. if |A| > m_max:
        return false
13. (tree_idx, leaf_idx) ← parse_idx(xof_out)
14. setLayerAddress(ADRS, 0)
15. setTreeAddress(ADRS, tree_idx[0] * 2^h_prime + leaf_idx[0])
16. pors_pk ← pors_pkFromSig(pors_sig, indices, PK.seed, ADRS)
17. msg ← pors_pk
18. xmss_sig_size ← R_SIZE + c + l * n + h_prime * n
19. for layer from 0 to d - 1:
    a. xmss_sig ← sig[offset : offset + xmss_sig_size]
    b. offset ← offset + xmss_sig_size
    c. wots_sig ← xmss_sig[0 : R_SIZE + c + l * n]
    d. auth ← xmss_sig[R_SIZE + c + l * n : R_SIZE + c + l * n + h' * n]
    e. setLayerAddress(ADRS, layer)
    f. setTreeAddress(ADRS, tree_idx[layer])
    g. msg ← xmss_pkFromSig(wots_sig, auth, msg, PK.seed, PK.root, ADRS, h_prime, leaf_idx[layer
```

```
        h. if msg == false:
               return false
    20. PK.sl ← msg
    21. setLayerAddress(ADRS, 0)
    22. setTreeAddress(ADRS, 0)
    23. setTypeAndClear(ADRS, ROOT)
    24. expected_root ← Tw(PK.seed, ADRS, PK.sf || PK.sl)
    25. return (expected_root == PK.root)
```

10.8 Unified Verification

The `SHRINCS.Verify` function provides a single entry point for verifying either stateful or stateless signatures. It distinguishes between the two types based on signature length: stateful signatures have a maximum size of `max_sf_size = n + R_SIZE + c + l*n + hsf*n` bytes (both `q = hsf` and `q = hsf + 1` produce hsf auth-path nodes), while stateless signatures are strictly larger (see Section 8.7 for the derivation that this bound always holds).

```
Algorithm: SHRINCS.Verify(m, sig, PK)
Input:
    m: message
    sig: signature
    PK: public key
Output:
    valid: boolean

1. max_sf_size ← n + R_SIZE + c + l * n + hsf * n
2. if len(sig) <= max_sf_size:
        return SHRINCS.VerifyStateful(m, sig, PK)
    else:
        return SHRINCS.VerifyStateless(m, sig, PK)
```

11 Data structures

This section specifies the binary encoding of SHRINCS keys and signatures. All multi-byte integers are encoded in big-endian byte order unless otherwise specified.

11.1 Secret Key Format

The secret key consists of six `n`-byte values:

```
SK.seed     : n bytes   // Seed for key derivation
SK.prf      : n bytes   // Secret for PRF_msg
PK.seed     : n bytes   // Domain separator
PK.sf       : n bytes   // Stateful public key
PK.sl       : n bytes   // Stateless public key
PK.root     : n bytes   // Combined public key
```

Total size: `6n = 96 bytes`. The first `3n` bytes (`SK.seed || SK.prf || PK.seed`) constitute the master seed that can be backed up using standard seed phrase mechanisms. The remaining `3n` bytes can be deterministically recomputed from the seed.

11.2 Public Key Format

The public key consists of two `n`-byte values:

```
PK.seed     : n bytes   // Domain separator
PK.root     : n bytes   // Root public key
```

Total size: `2n = 32 bytes`.

11.3 Stateful Signature Format

A stateful signature consists of the stateless public key followed by a UXMSS signature:

```
PK.sl       : n bytes                 // Stateless public key
R           : R_SIZE bytes            // Randomness for message hashing
ctr         : c bytes                 // Grinding counter
chain_values: l * n bytes             // WOTS+C signature
auth_path   : min(q, hsf) * n bytes   // UXMSS authentication path
```

The signature size depends on the signature index `q`:

```
SHRINCS-B (n=16, R_SIZE=32, c=4, l=16):
    PK.sl       : 16 bytes
    R           : 32 bytes
    ctr         : 4 bytes
    chain_values: 256 bytes
    auth_path   : min(q, hsf) * 16 bytes

    Total: 308 + 16*min(q, hsf) bytes

SHRINCS-L (n=16, R_SIZE=32, c=4, l=64):
    PK.sl       : 16 bytes
    R           : 32 bytes
    ctr         : 4 bytes
    chain_values: 1024 bytes
    auth_path   : min(q, hsf) * 16 bytes

    Total: 1076 + 16*min(q, hsf)
```

11.4 Stateless Signature Format

A stateless signature consists of the stateful public key, a PORS+FP signature, and `d` XMSS signatures forming the hypertree:

```
PK.sf    : n bytes                    // Stateful public key
PORS+FP signature:
  pors_R   : R_SIZE bytes             // Randomness (encodes grinding result)
  pors_sigs: (k+m_max)*n bytes        // k revealed secret keys
Hypertree: d XMSS signatures. For each layer i:
  xmss_R[i]     : R_SIZE bytes        // Randomness for WOTS+C
  xmss_ctr[i]   : c bytes             // Grinding counter for WOTS+C
  xmss_chains[i]: l * n bytes         // WOTS+C chain values
  xmss_auth[i]  : h_prime * n bytes   // XMSS authentication path

SHRINCS-B (n=16, R_SIZE=32, c=4, k=6, m_max = 91, d=2, h'=12, l=16):
  PK.sf          :   16 bytes
  PORS+FP signature:
    pors_R       :   32 bytes
    pors_sigs    : (6 + 91) * 16 = 97 * 16 = 1552 bytes
    subtotal     : 1584 bytes
  XMSS hypertree (2 layers):
    per layer    : 32 + 4 + 256 + 192 = 484 bytes
    2 layers     : 968 bytes
  Total: 16 + 1584 + 968 = 2568 bytes

SHRINCS-L (n=16, R_SIZE=32, c=4, k=6, m_max = 91, d=2, h'=12, l=64):
  PK.sf          :   16 bytes
```

```
PORS+FP signature:
  pors_R       :   32 bytes
  pors_sigs    : (6 + 91) * 16 = 97 * 16 = 1552 bytes
  subtotal     : 1584 bytes
XMSS hypertree (2 layers):
  per layer    : 32 + 4 + 1024 + 192 = 1252 bytes
  2 layers     : 2504 bytes
Total: 16 + 1584 + 2504 = 4104 bytes
```

11.5 Signature Type Indication

Verifiers distinguish between stateful and stateless signatures by length (Algorithm 10.8):

```
max_stateful_size = n + R_SIZE + c + l*n + hsf*n
```

- For SHRINCS-B: `max_stateful_size = 16 + 32 + 4 + 256 + 141*16 = 2564` bytes
- For SHRINCS-L: `max_stateful_size = 16 + 32 + 4 + 1024 + 189*16 = 4100` bytes

Signatures with length `<= max_stateful_size` are processed as stateful; larger signatures as stateless.

References

[AK25] Mehdi Abri and Jonathan Katz. *Shorter Hash-Based Signatures Using Forced Pruning*. Cryptology ePrint Archive, Paper 2025/2069. 2025. URL: https://eprint.iacr.org/2025/2069.

[AE18] Jean-Philippe Aumasson and Guillaume Endignoux. "Improving Stateless Hash-Based Signatures". In: *Topics in Cryptology – CT-RSA 2018*. Vol. 10808. Lecture Notes in Computer Science. Springer, 2018, pp. 219–242. DOI: 10.1007/978-3-319-76953-0_12.

[Ber+19] Daniel J. Bernstein et al. *The SPHINCS+ Signature Framework*. Full version. Sept. 2019. URL: https://sphincs.org/data/sphincs+-paper.pdf.

[Coo+20] David A. Cooper et al. *Recommendation for Stateful Hash-Based Signature Schemes*. NIST Special Publication 800-208. Oct. 2020. DOI: 10.6028/NIST.SP.800-208. URL: https://csrc.nist.gov/pubs/sp/800/208/final.

[Hül13] Andreas Hülsing. "W-OTS+ – Shorter Signatures for Hash-Based Signature Schemes". In: *Progress in Cryptology – AFRICACRYPT 2013*. Vol. 7918. Lecture Notes in Computer Science. Springer, 2013, pp. 173–188. DOI: 10.1007/978-3-642-38553-7_10. URL: https://link.springer.com/chapter/10.1007/978-3-642-38553-7_10.

[KN25] Mikhail Kudinov and Jonas Nick. *Hash-Based Signatures for Bitcoin's Post-Quantum Future*. IACR Cryptology ePrint Archive, Report 2025/2203. Technical report referenced on the Bitcoin Development Mailing List. Dec. 2025. URL: https://eprint.iacr.org/2025/2203.

[Kud+22] Mikhail Kudinov et al. *SPHINCS+C: Compressing SPHINCS+ With (Almost) No Cost*. IACR Cryptology ePrint Archive, Report 2022/778. Introduces WOTS+C (a compressed variant of WOTS+) and FORS+C. 2022. URL: https://eprint.iacr.org/2022/778.

[Nat13] National Institute of Standards and Technology. *Digital Signature Standard (DSS)*. Federal Information Processing Standards Publication (FIPS PUB) 186-4. Includes the Elliptic Curve Digital Signature Algorithm (ECDSA). July 2013. URL: https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.186-4.pdf.

[Nat24] National Institute of Standards and Technology. *Stateless Hash-Based Digital Signature Standard*. Federal Information Processing Standards Publication (FIPS PUB) 205. Specifies SLH-DSA (standardized SPHINCS+). Aug. 2024. DOI: 10.6028/NIST.FIPS.205. URL: https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.205.pdf.

[Pal+13] Marek Palatinus et al. *BIP 39: Mnemonic code for generating deterministic keys*. Bitcoin Improvement Proposal (BIP) 39. Status: Proposed. Sept. 2013. URL: https://bips.dev/39/.

[Sho94] Peter W. Shor. "Algorithms for Quantum Computation: Discrete Logarithms and Factoring". In: *Proceedings of the 35th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 1994, pp. 124–134. DOI: 10.1109/SFCS.1994.365700. URL: https://doi.org/10.1109/SFCS.1994.365700.

[Sho97] Peter W. Shor. "Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer". In: *SIAM Journal on Computing* 26.5 (1997), pp. 1484–1509. DOI: 10.1137/S0097539795293172. URL: https://doi.org/10.1137/S0097539795293172.

[Wig+26] Thom Wiggers et al. *Hash-based Signatures: State and Backup Management*. Internet-Draft (work in progress), draft-ietf-pquip-hbs-state-03. Expires 11 August 2026. Feb. 2026. URL: https://datatracker.ietf.org/doc/draft-ietf-pquip-hbs-state/.

[WNR20] Pieter Wuille, Jonas Nick, and Tim Ruffing. *BIP 340: Schnorr Signatures for secp256k1*. Bitcoin Improvement Proposal (BIP) 340. Status: Deployed. Jan. 2020. URL: https://bips.dev/340/.

A Expected Grinding Complexity

This section analyzes the computational cost of the grinding operations in WOTS+C and PORS+FP for both SHRINCS parameter sets.

A.1 Probabilistic Analysis

A.1.1 Rejection Sampling

Rejection sampling used to obtain k distinct indices from the set $\{0, 1, ..., t-1\}$ in `pors_digest_to_indices` function, where t is an arbitrary number that is not the power of 2. Inside of `pors_digest_to_indices` we evaluate some number of SHA256 blocks, and sequentialy extract $b = \lceil \log_2(t) \rceil$ bits for every index, which we will call a *candidate*. A candidate that is distinct among previous candidates and is in $[0, t-1]$ is "good". The extraction of next candidate is independent from previous draws index. Moreover, each candidate chosen uniformly from $\{0, 1, ..., 2^b - 1\}$, and rejetion smapling does not affect the uniformity of choosing a candidate.

Let's define probability that extracted i-th index is distinct:

$$p_c^{(i)} = \frac{t - (i-1)}{t} \tag{3}$$

and define the probability that the obtained index will be in range $[0, t-1]$:

$$p_r = \frac{t}{2^b} \tag{4}$$

Then the probability of obtaining "good" i-th candidate is simply the product of $p_c^{(i)} \cdot p_r$. Then

$$p_i = \frac{t - (i-1)}{t} \cdot \frac{t}{2^b} = \frac{t - (i-1)}{2^b} \tag{5}$$

As we can see, each candidate draw is the Bernoulli trial with probability of success p_i. Furthermore, each draw of good candidate obeys the geometric distribution with success probability p_i and expected number of candidates $1/p_i$. The expected total number of candidates to obtain k "good" is:

$$\mathbb{E}[\text{total candidates}] = \sum_{i=1}^{k} \frac{1}{p_i} = 2^b \sum_{i=1}^{k} \frac{1}{t - (i-1)} \tag{6}$$

If $k \ll t$, then we can aproximate the expression (6) as:

$$\mathbb{E}[\text{total candidates}] \approx 2^b \cdot \frac{k}{t} \tag{7}$$

The expected number of hashes during rejection smapling is the following:

$$\mathbb{E}[\text{rejection sampling hashes}] = \left\lceil \frac{\mathbb{E}[\text{total candidates}]}{\lfloor \frac{256}{b} \rfloor} \right\rceil \tag{8}$$

A.1.2 WOTS+C Grinding

The grinding operation searches for a counter `ctr` such that the sum of base-w digits equals the target S_{wn}. Each digest attempt produces l independent random digits, each uniformly distributed in $[0, w-1]$.

For a single digit:

$$\mathbb{E}[d_i] = \frac{w-1}{2} \tag{9}$$

$$\text{Var}[d_i] = \frac{w^2-1}{12} \tag{10}$$

For the sum of l digits:

$$\mathbb{E}\left[\sum d_i\right] = l \cdot \frac{w-1}{2} \tag{11}$$

$$\text{Var}\left[\sum d_i\right] = l \cdot \frac{w^2-1}{12} \tag{12}$$

$$\sigma = \sqrt{l \cdot \frac{w^2-1}{12}} \tag{13}$$

By the Central Limit Theorem, for moderate l, the sum distribution approximates:

$$\sum d_i \sim \mathcal{N}\left(\mu = l \cdot \frac{w-1}{2}, \sigma^2\right) \tag{14}$$

The probability of hitting exactly S_{wn} can be approximated as:

$$\Pr\left(\sum d_i = S_{wn}\right) \approx \frac{1}{\sigma\sqrt{2\pi}} \cdot \exp\left(-\frac{(S_{wn}-\mu)^2}{2\sigma^2}\right) \tag{15}$$

Expected grinding iterations:

$$\mathbb{E}[\text{iterations}] = \frac{1}{\Pr(\sum d_i = S_{wn})} \tag{16}$$

A.1.3 PORS+FP Grinding

The grinding operation searches for $|A| \leq m_{\max}$. Let $|A| = S(t,k)$. Then the expected number of iterations during grinding is the following:

$$\Pr(|A| \leq m_{\max}) = \sum_{m=1}^{m_{\max}} \Pr(S(t,k) = m) \tag{17}$$

$$\mathbb{E}[\text{iterations}] = \frac{1}{\Pr\left(|A| \leq m_{\max}\right)} \tag{18}$$

where $\Pr(S(t,k) = m)$ is the probability that the size of authentication set is exactly m. The analytical expression for this probability descibed and derived in [AK25]. We omit to describe the formula directly, due to complex derivation.

We also need to include the rejection sampling hash invocations (Apendix 1.1), which is inside the grinding loop, resulting in the total expected hash operations:

$$\mathbb{E}[\text{hashes}] = \mathbb{E}[\text{iterations}] \cdot \mathbb{E}[\text{rejection sampling hashes}] \tag{19}$$

A.2 Concrete Parameters

A.2.1 SHRINCS-B ($w = 256$, $l = 16$, $S_{wn} = 2040$)

Statistic calculation:

$$\mu = 16 \cdot \frac{255}{2} = 2040 \tag{20}$$

$$\sigma^2 = 16 \cdot \frac{256^2 - 1}{12} \approx 87381 \tag{21}$$

$$\sigma \approx 295.6 \tag{22}$$

Analysis:

- $S_{wn} = 2040$ is exactly at the expected value ($\mu = 2040$)
- Distance from mean: $(2040 - 2040)/295.6 = 0$ standard deviations

Approximate probability:

$$\Pr\left(\sum d_i = 2040\right) \approx \frac{1}{295.6 \cdot \sqrt{2\pi}} \approx \frac{1}{741} \tag{23}$$

Expected grinding cost:

- $\mathbb{E}[\text{iterations}] \approx 741$ iterations
- Per iteration: 2 SHA-256 compression (Phase 2 operates on 68-byte input)
- Total: ≈ 741 SHA-256 compressions

Practical performance:

- $\approx$1-2 million SHA-256/sec
- Expected grinding time: $<$1 millisecond

A.2.2 SHRINCS-L ($w = 4$, $l = 64$, $S_{wn} = 140$)

Statistic calculation:

$$\mu = 64 \cdot \frac{3}{2} = 96 \tag{24}$$

$$\sigma^2 = 64 \cdot \frac{4^2 - 1}{12} = 80 \tag{25}$$

$$\sigma \approx 8.94 \tag{26}$$

Analysis:

- $S_{wn} = 140$ is significantly above the expected value
- Distance from mean: $(140 - 96)/8.94 \approx 4.92$ standard deviations

Approximate probability:

$$\begin{aligned}\Pr\left(\sum d_i = 140\right) &\approx \frac{1}{8.94\sqrt{2\pi}} \cdot \exp\left(-\frac{4.92^2}{2}\right) \\ &\approx \frac{1}{22.4} \cdot \exp(-12.1) \\ &\approx \frac{1}{22.4} \cdot 5.5 \times 10^{-6} \\ &\approx 2.45 \times 10^{-7}\end{aligned} \tag{27}$$

Expected grinding cost:

- $\mathbb{E}[\text{iterations}] \approx 4.1\text{m}$ iterations
- Per iteration: 1 SHA-256 compression
- Total: ≈ 4.1 million SHA-256 compressions

Practical performance:

- $\approx$1-2 million SHA-256/sec
- Expected grinding time: 2-4 seconds per signature

A.3 PORS+FP Grinding Complexity

Both SHRINCS-B and SHRINCS-L use identical PORS+FP parameters:

- $k = 6$ trees
- $t = 9245141$ (number of leaves)
- $m_{\max} = 91$

  Grinding condition: $|A| \leq 91$

Expected grinding cost:

- $\mathbb{E}[\text{iterations}] = 2526130 \cdot 2 = 505226\text{m}$ iterations
- Per iteration: 1 HMAC-SHA-256 + 3 SHA-256

Practical performance:

- $\sim$1-2 million SHA-256/sec
- Expected grinding time: 4-8 seconds per stateless signature